Отчёт по лабораторной работе №2

Иммитационное моделирование

Арбатова Варвара Петровна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Рабочее пространство 8

4.2 Модель SIR 9

4.3 Модель Лотки-Вольтерры 30

5 Выводы **54**

Список литературы **55**

Список иллюстраций

4.1 Созданное рабочее пространство 8
4.2 Скачивание библиотек 8
4.3 Анализ результатов 28
4.4 Графики SIR 29
4.5 Создание файлов 29
4.6 30
4.7 Анализ результатов 52
4.8 Анализ чувствительности к параметрам 53
4.9 Графики 53

Список таблиц

1 Цель работы

Выполнить предложенный код и изучить, как работают модели SIR и Лотки-Вольтерры

2 Задание

— Создать рабочий каталог для кода. — Установить необходимые пакеты. — Выполнить предложенный код. — Преобразовать код в литературный стиль. — Сгенерировать из литературного кода: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook. — Интегрировать документацию в формате Quarto в отчёт. — Добавить в код в литературном стиле вычисление для набора параметров. — Сгенерировать из литературного кода с параметрами: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook с параметрами. — Интегрировать документацию с параметрами в формате Quarto в отчёт

3 Теоретическое введение

Модель SIR есть классическая и фундаментальная математическая модель эпиде- миологии, описывающая распространение инфекционного заболевания в закры- той популяции [2; 3]

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии: — S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться. — I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию. — R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи. Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться, как быстро, сколько людей в итоге переболеет, как влияют карантинные меры.

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв. Она была независимо предложена в 1920-х годах: — Альфредом Лоткой (1925) для химических реакций [5; 6]. — Витторио Вольтеррой (1926) для объяснения колебаний улова рыбы в Адриатическом море [8; 9]. Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические изменения численности в природных экосистемах.

4 Выполнение лабораторной работы

4.1 Рабочее пространство

Создаю рабочее пространство по гайду из первой лабораторной работы (рис. 4.1).

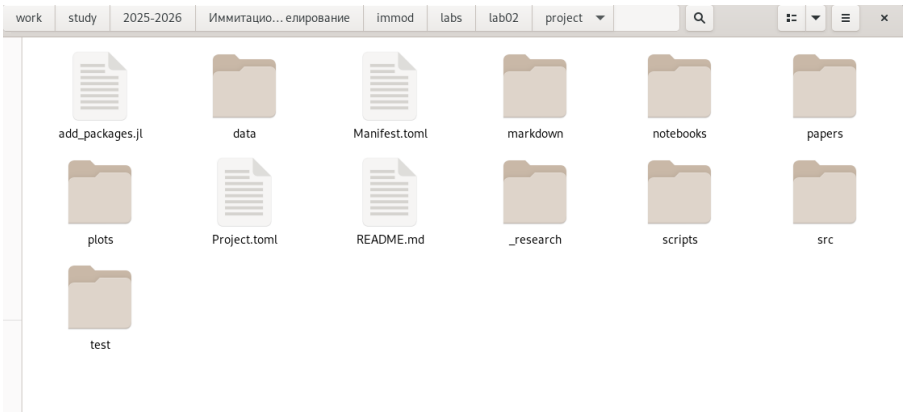


Рисунок 4.1: Созданное рабочее пространство

Скачиваю недостающие библиотеки для лабораторной работы (рис. 4.2).

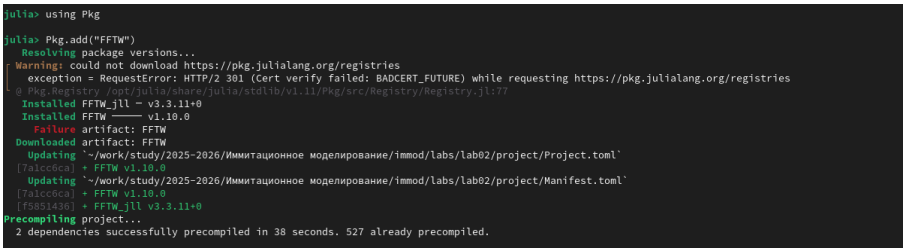


Рисунок 4.2: Скачивание библиотек

4.2 Модель SIR

```
using DrWatson
@quickactivate "project"
using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings # Для красивого отображения формул на графиках
using Plots
using BenchmarkTools
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
function sir_ode!(du, u, p, t)
(S, I, R) = u
(β, c, γ) = p
N = S + I + R
@inbounds begin
du[1] = -β * c * I / N * S
du[2] = β * c * I / N * S - γ * I
du[3] = γ * I
end
nothing
end
```

```
  Activating project at `~/work/study/2025-2026/Иммитационное моделирование
```

```
sir_ode! (generic function with 1 method)
```

Параметры модели

```julia
δt = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0] # S, I, R
p = [0.05, 10.0, 0.25] # β, c, γ
```

```
3-element Vector{Float64}:
  0.05
 10.0
  0.25
```

Расчет базового репродуктивного числа

```julia
R0 = (p[2] * p[1]) / p[3] # R0 = (c * β) / γ
```

```
2.0
```

Создание и решение задачи

```julia
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = δt)
```

```
retcode: Success
Interpolation: 3rd order Hermite
t: 15-element Vector{Float64}:
  0.0
  0.1
  0.5331242537191838
  1.3919009080055345
  2.61021221798537
  4.181182326044298
  6.180124045228886
  8.664952146237743
```

```
 11.691425523605613
 15.27937105275849
 19.0829636767576
 23.423425969211983
 28.429494244539086
 33.29877943214063
 40.0
u: 15-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0]
 [989.4990153200853, 10.247898072276143, 0.2530866076385313]
 [987.1852964117763, 11.391111426852984, 1.4235921613706282]
 [981.8312121172252, 14.026022241732, 4.142765641042832]
 [972.139229760034, 18.757816192500677, 9.102954047465346]
 [954.9905468224117, 27.007704055602183, 18.001749121986144]
 [923.070667251774, 41.929769879700146, 34.99956286852594]
 [862.6705746724687, 68.49345686513698, 68.83596846239446]
 [754.3022914470604, 109.7418645302877, 135.95584402265195]
 [595.101409623216, 150.41363540416995, 254.4849549726141]
 [442.01411154679107, 154.80694904937002, 403.17893940383897]
 [326.895101656965, 119.0685959489173, 554.0363023941178]
 [258.5085195746908, 70.11051759777406, 671.3809628275352]
 [227.58338228204383, 37.328569316974345, 735.0880484009818]
 [209.84121402510556, 14.473644966335414, 775.6851410085591]
```

Подготовка данных в DataFrame

```
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R # Общая численность популяци
```

```
15-element Vector{Float64}:
```

```
 1000.0
 1000.0
  999.9999999999999
 1000.0
 1000.0
 1000.0
 1000.0000000000001
 1000.0000000000001
 1000.0
 1000.0
 1000.0
 1000.0000000000001
 1000.0
 1000.0
 1000.0
```

Вывод параметров модели

```
println("Параметры модели SIR:")
println("β (вероятность заражения) = ", p[1])
println("c (среднее число контактов) = ", p[2])
println("γ (скорость выздоровления) = ", p[3])
println("R0 = c * β / γ = ", round(R0, digits=3))
println("Средняя продолжительность болезни = ", round(1/p[3],
digits=2), " дней")
println("Начальные условия: S0 = ", u0[1], ", I0 = ", u0[2], ", R0 =
", u0[3])
```

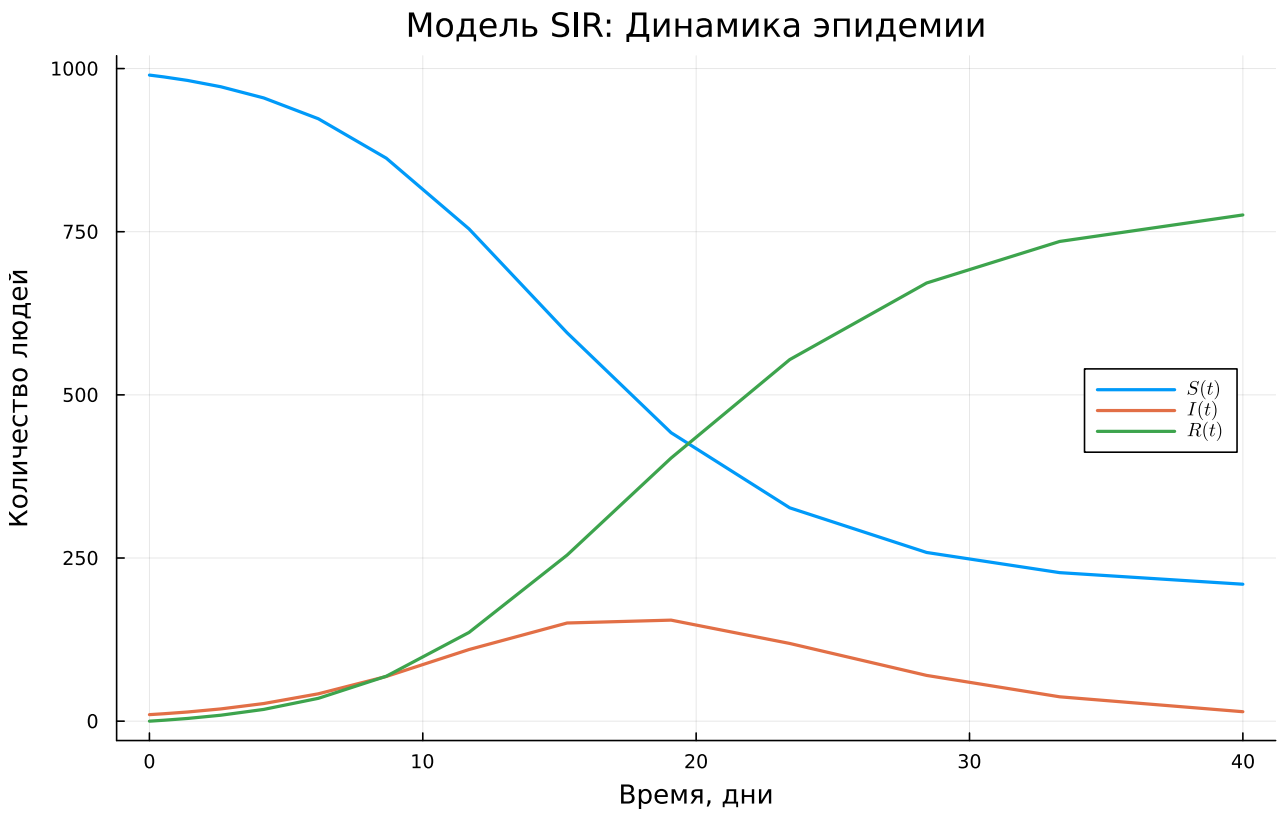
```
Параметры модели SIR:
β (вероятность заражения) = 0.05
c (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
```

```
R0 = c * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 =
0.0
```

1. ОСНОВНОЙ ГРАФИК: динамика всех трех групп

```
plt1 = @df df_ode plot(:t,
[:S :I :R],
label=[L"S(t)" L"I(t)" L"R(t)"],
xlabel="Время, дни",
ylabel="Количество людей",
title="Модель SIR: Динамика эпидемии",
linewidth=2,
legend=:right,

grid=true,
size=(800, 500))
```



Добавление аннотаций с параметрами

```
annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
text("Параметры:\nβ = $(p[1])\nc = $(p[2])\nγ = $(p[3])\nR0 =
$(round(R0, digits=2))",
8, :left))
```

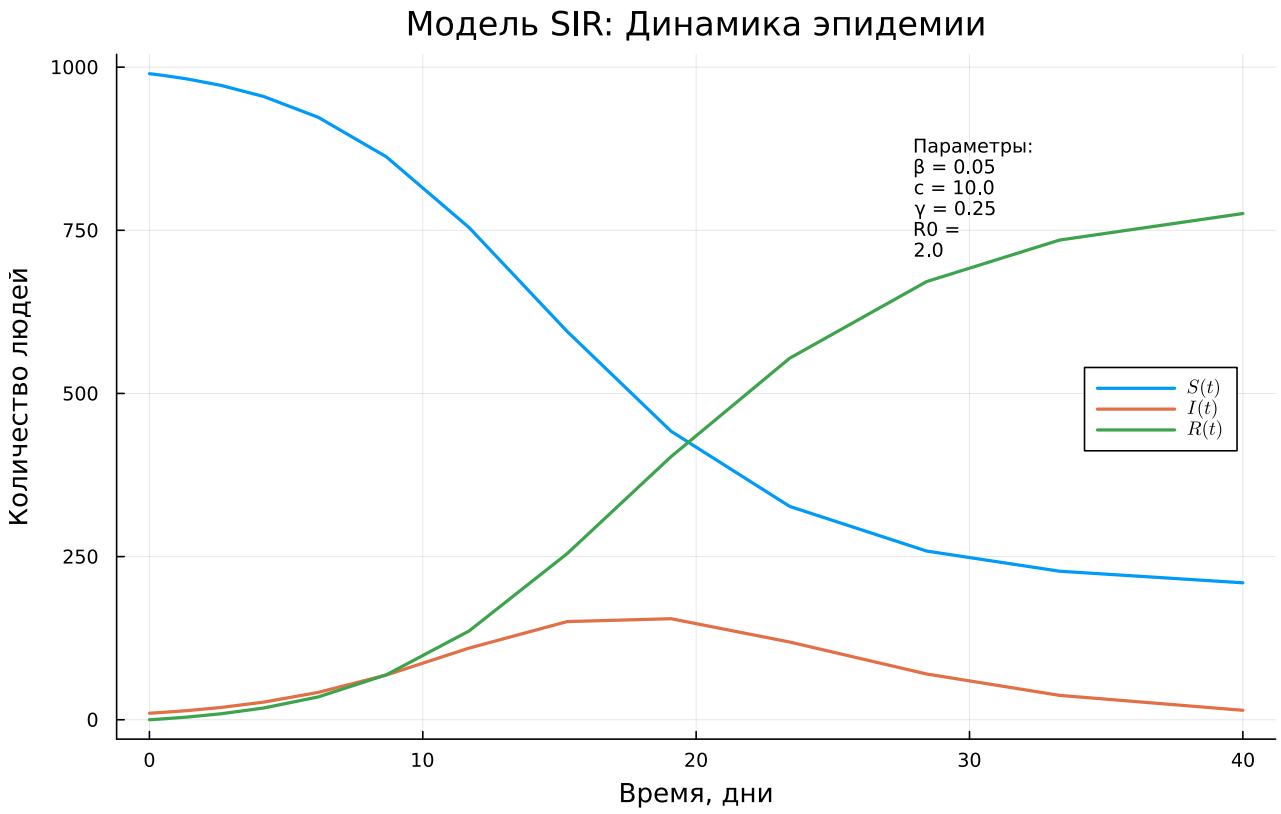
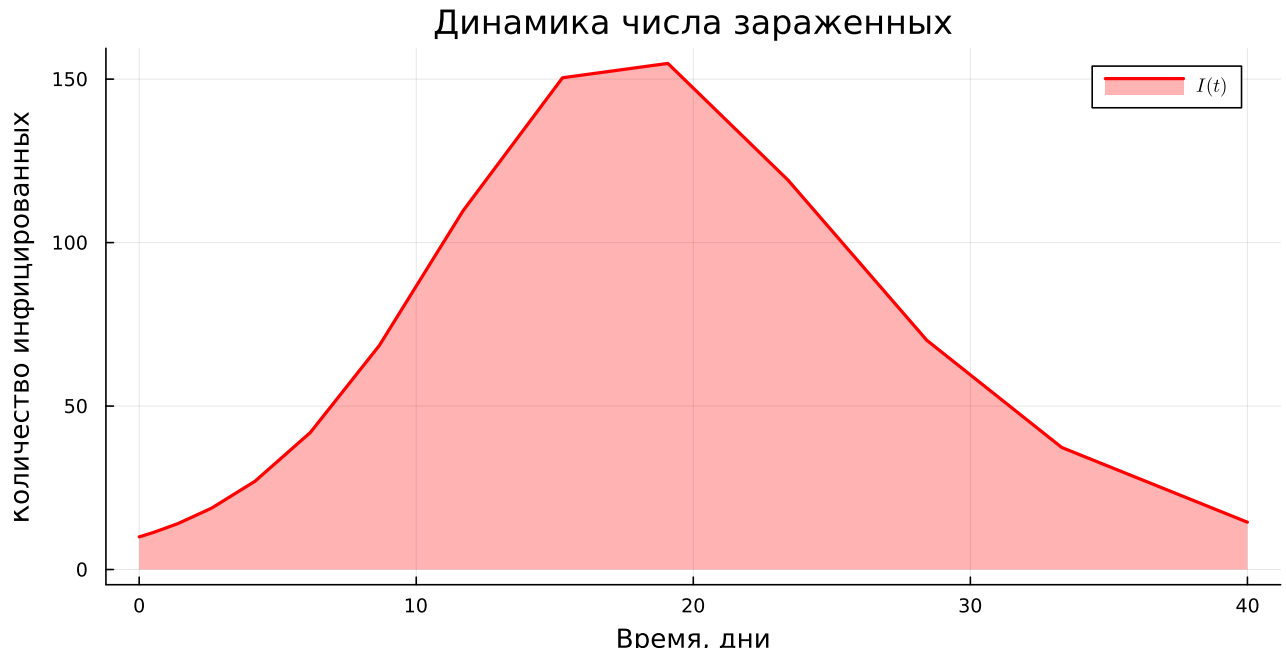


График только инфицированных (I)

```
plt2 = @df df_ode plot(:t, :I,
label=L"I(t)",
xlabel="Время, дни",
ylabel="Количество инфицированных",
title="Динамика числа зараженных",
color=:red,
linewidth=2,
fill=(0, 0.3, :red),
grid=true,
```

```
size=(800, 400))
```



Отметка пика эпидемии

```
peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash,
label=false, linewidth=1)
annotate!(plt2, peak_time, peak_value * 1.05,
text("Пик: $(round(peak_value, digits=1)) на $(round(peak_time,
digits=1)) день",
8, :top))
```

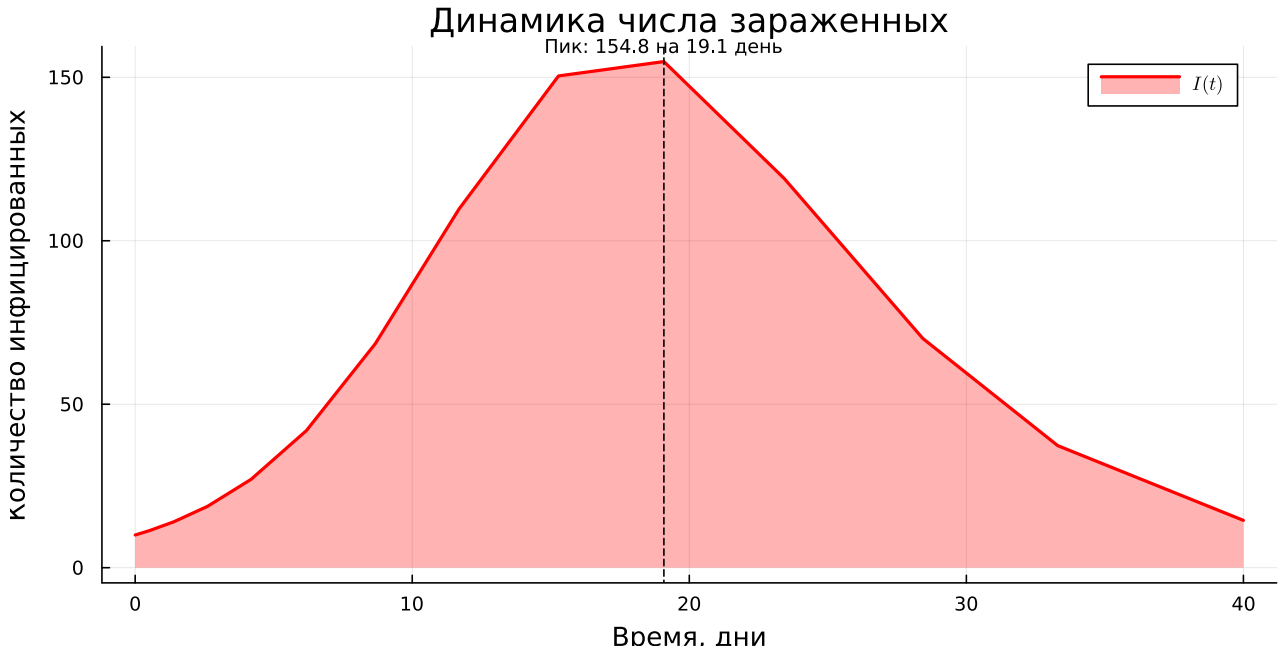


График в логарифмическом масштабе (для анализа экспоненциального роста)

```
plt3 = @df df_ode plot(:t, :I,
label=L"I(t)",
xlabel="Время, дни",
ylabel="Количество инфицированных (лог. масштаб)",
title="Экспоненциальный рост (лог. шкала)",
yscale=:log10,
color=:red,
linewidth=2,
grid=true,
size=(800, 400))
```

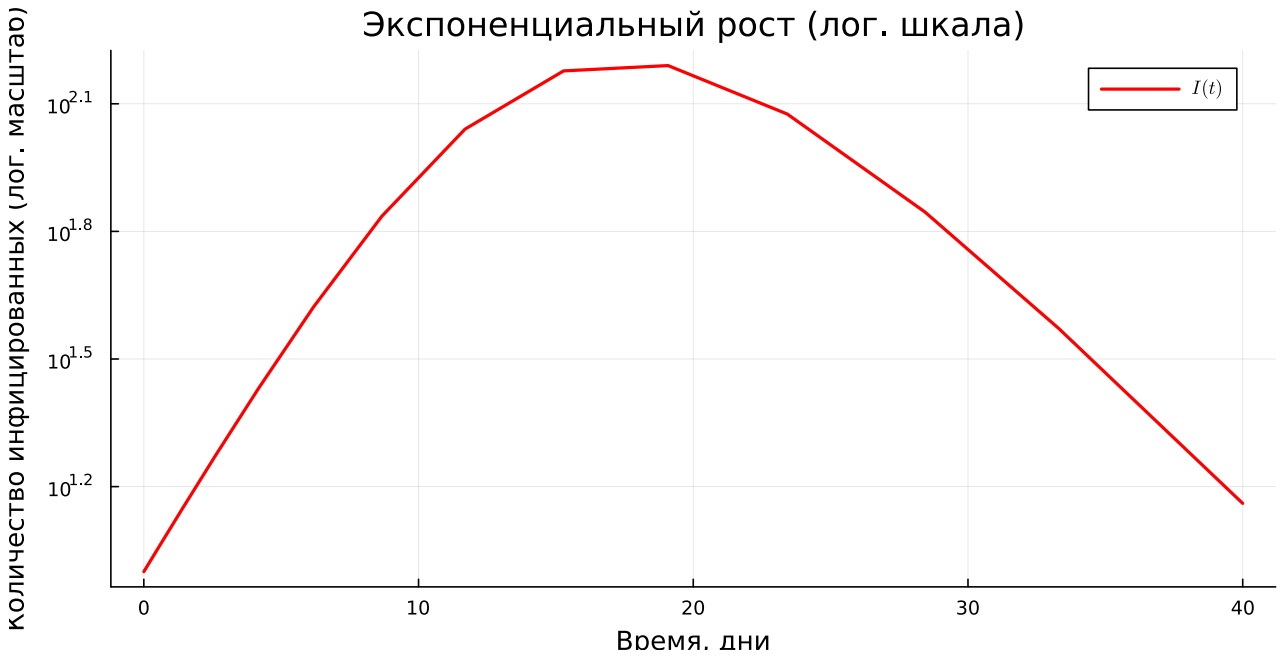
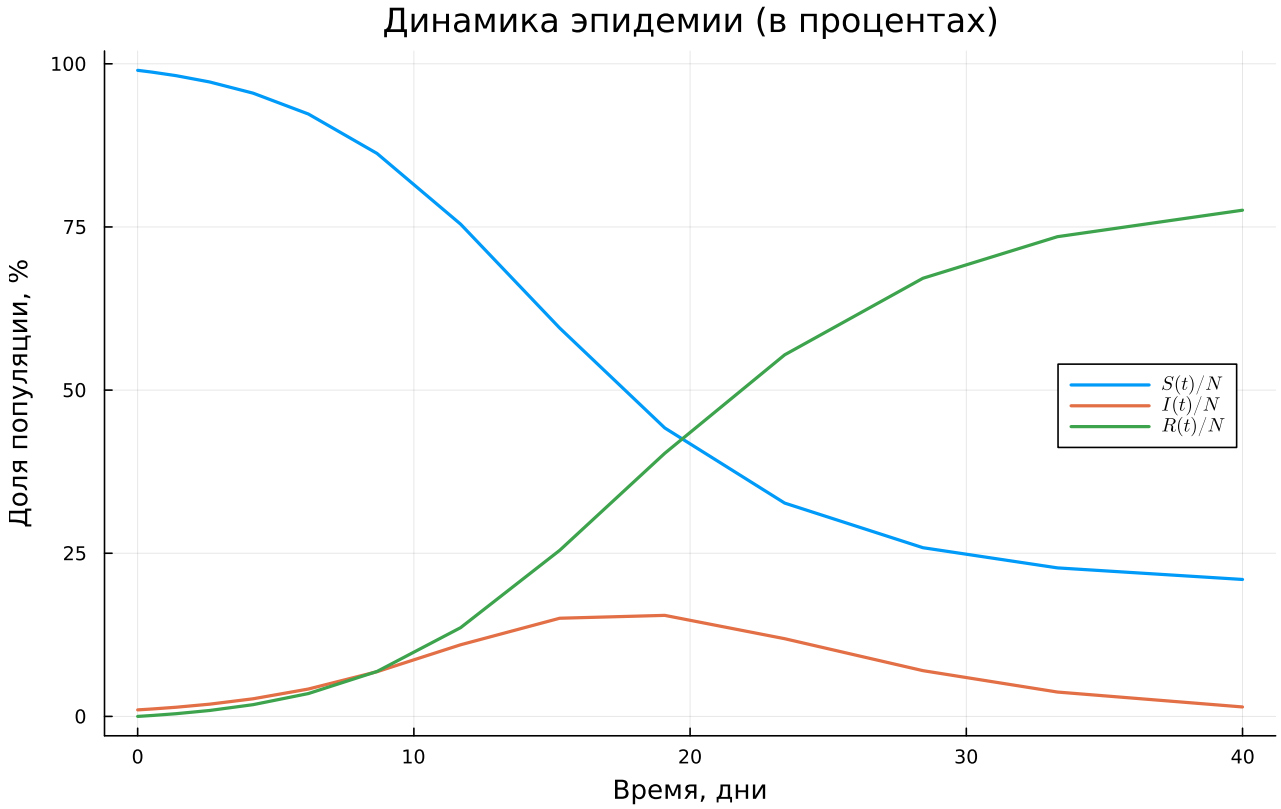


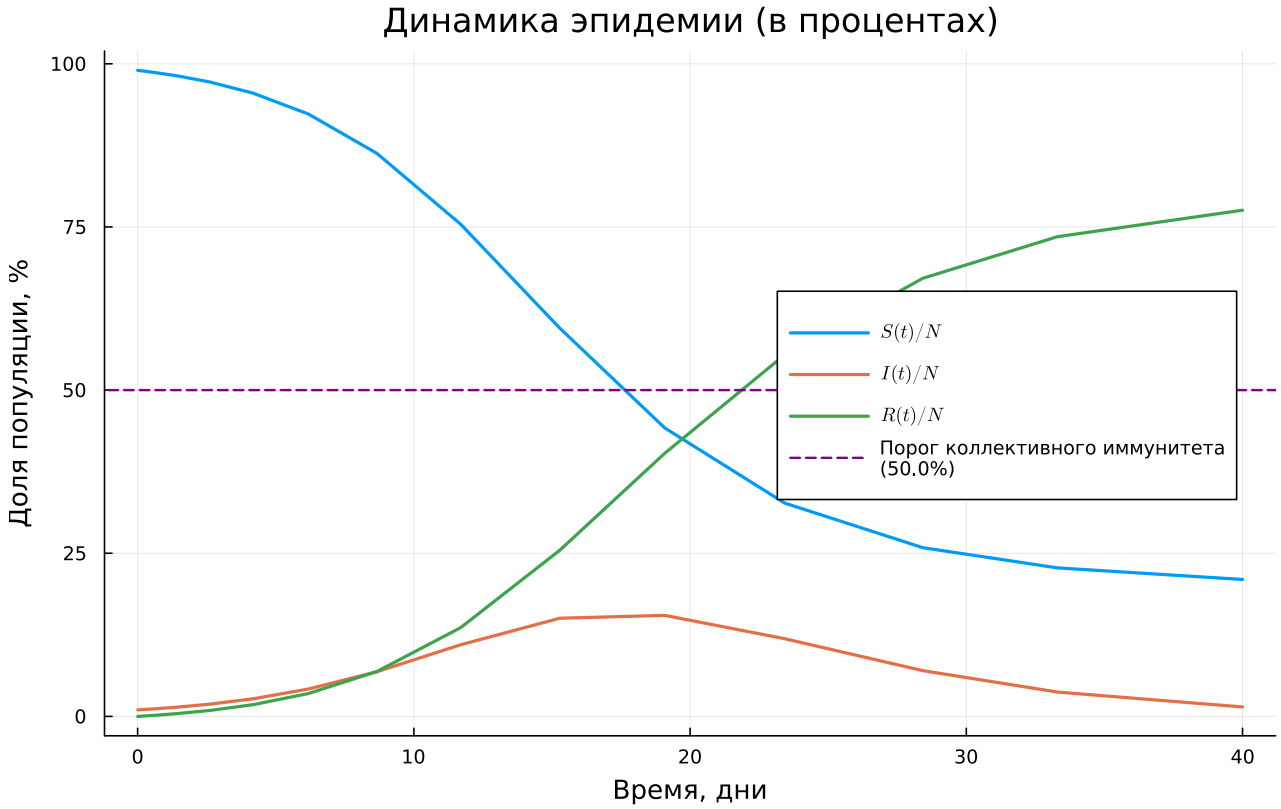
График долей населения (в процентах)

```
plt4 = @df df_ode plot(:t,
[:S :I :R] ./ df_ode.N .* 100,
label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
xlabel="Время, дни",
ylabel="Доля популяции, %",
title="Динамика эпидемии (в процентах)",
linewidth=2,
legend=:right,
grid=true,
size=(800, 500))
```



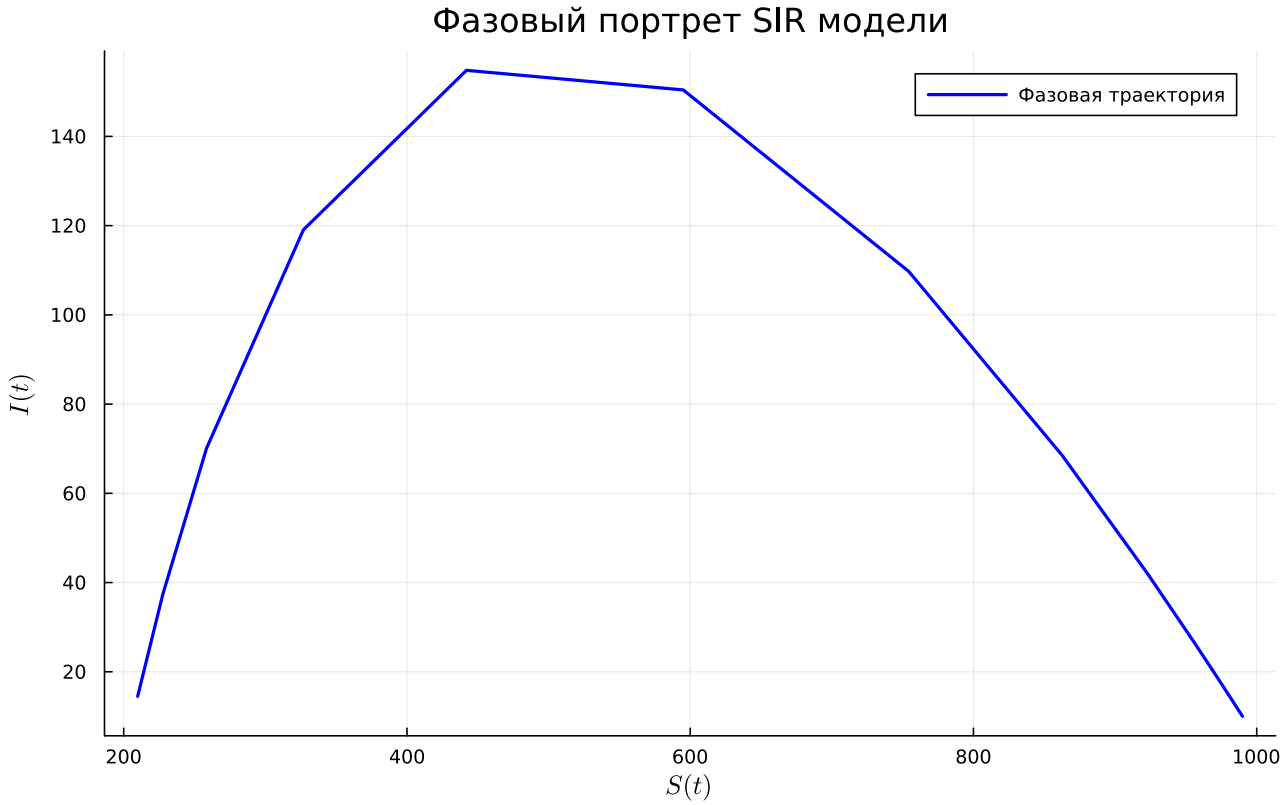
Горизонтальная линия для порога коллективного иммунитета

```
if R0 > 1
herd_immunity_threshold = (1 - 1/R0) * 100
hline!(plt4, [herd_immunity_threshold], color=:purple,
linestyle=:dash,
label="Порог коллективного иммунитета
($(round(herd_immunity_threshold, digits=1))%)",
linewidth=1.5)
end
```



Фазовый портрет (I vs S)

```
plt5 = plot(df_ode.S, df_ode.I,
label="Фазовая траектория",
xlabel=L"S(t)",
ylabel=L"I(t)",
title="Фазовый портрет SIR модели",
color=:blue,
linewidth=2,
grid=true,
size=(800, 500),
legend=:topright)
```



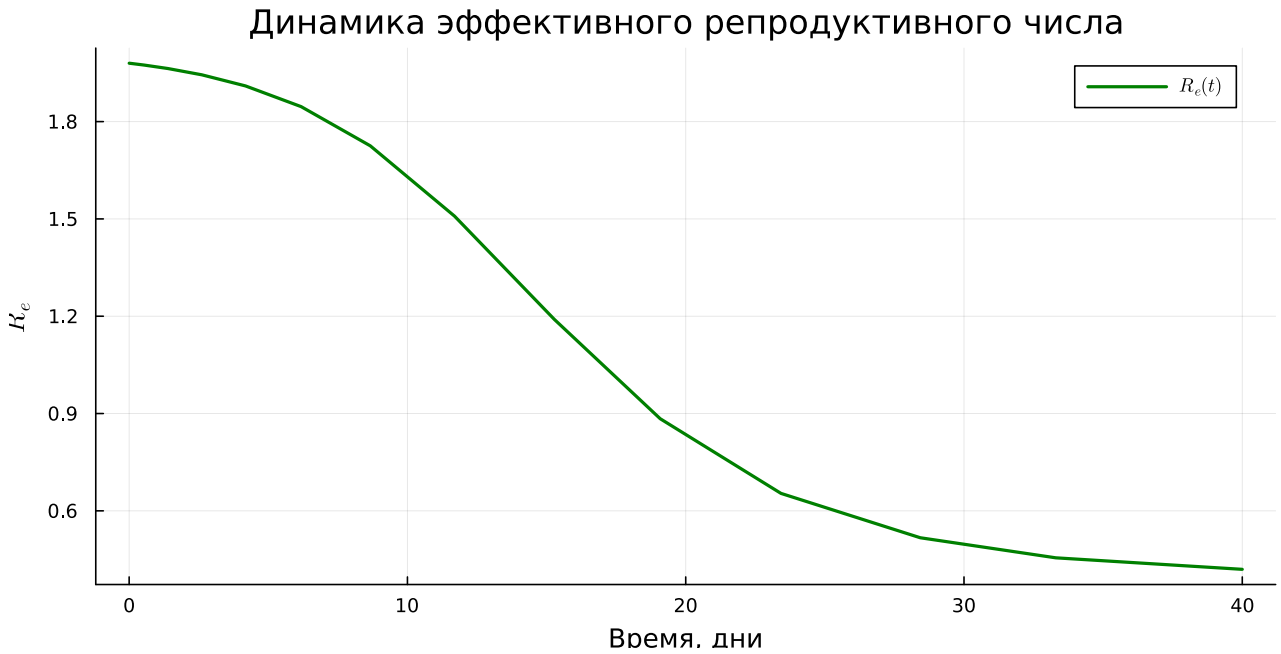
Добавление стрелок направления

```
for i in 1:50:length(df_ode.S)-1
plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i],
df_ode.I[i+1]],
arrow=:closed, color=:blue, alpha=0.5, label=false)
end
```

График R_e - эффективного репродуктивного числа

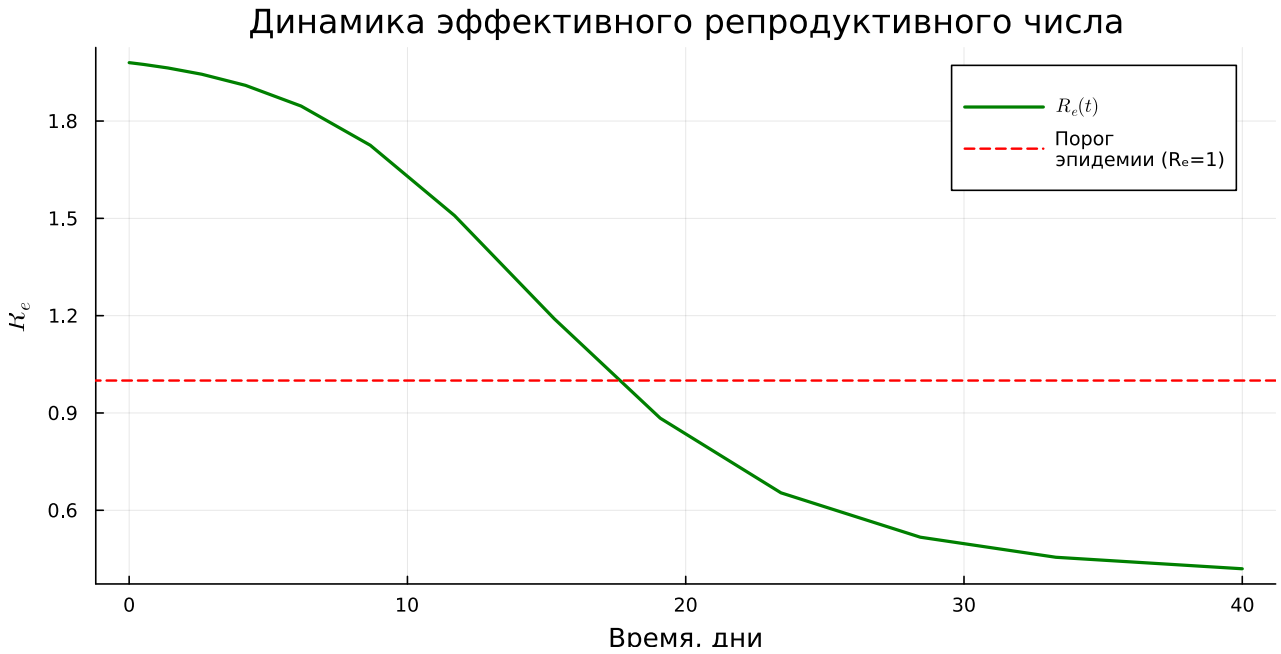
```
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
plt6 = @df df_ode plot(:t, :Re,
label=L"R_e(t)",
xlabel="Время, дни",
ylabel=L"R_e",
title="Динамика эффективного репродуктивного числа",
color=:green,
linewidth=2,
```

```
grid=true,
size=(800, 400))
```



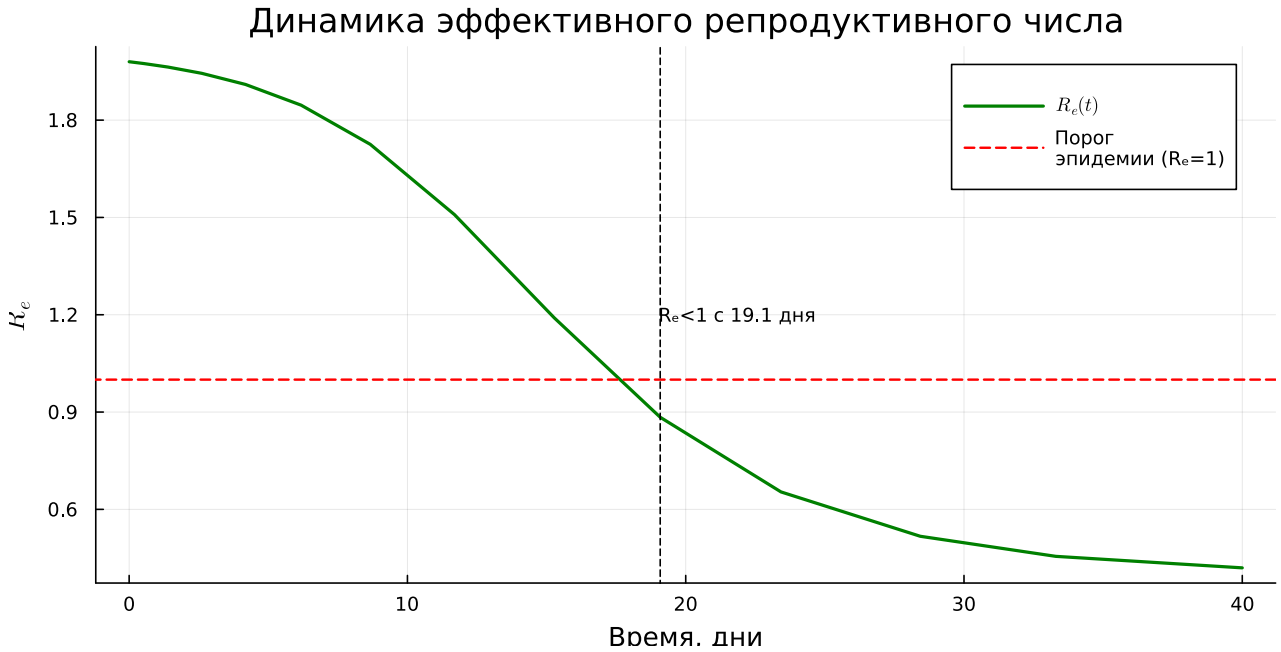
Горизонтальная линия на уровне 1

```
hline!(plt6, [1.0], color=:red, linestyle=:dash, label="Порог
эпидемии (Re=1)", linewidth=1.5)
```



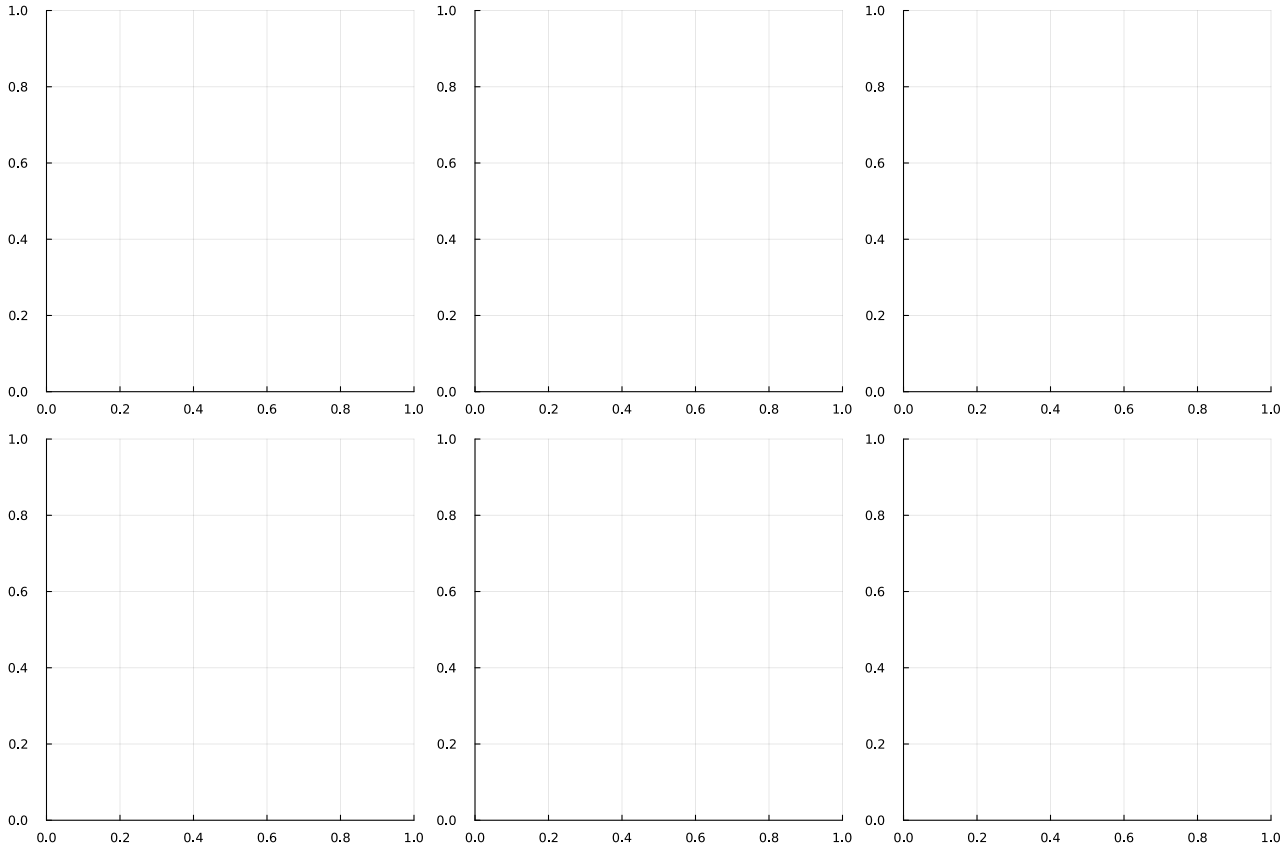
Отметка момента, когда R_e становится < 1

```
cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
cross_time = df_ode.t[cross_idx]
vline!(plt6, [cross_time], color=:black, linestyle=:dash,
label=false, linewidth=1)
annotate!(plt6, cross_time, 1.2,
text("Rₑ<1 с $(round(cross_time, digits=1)) дня", 8, :left))
end
```



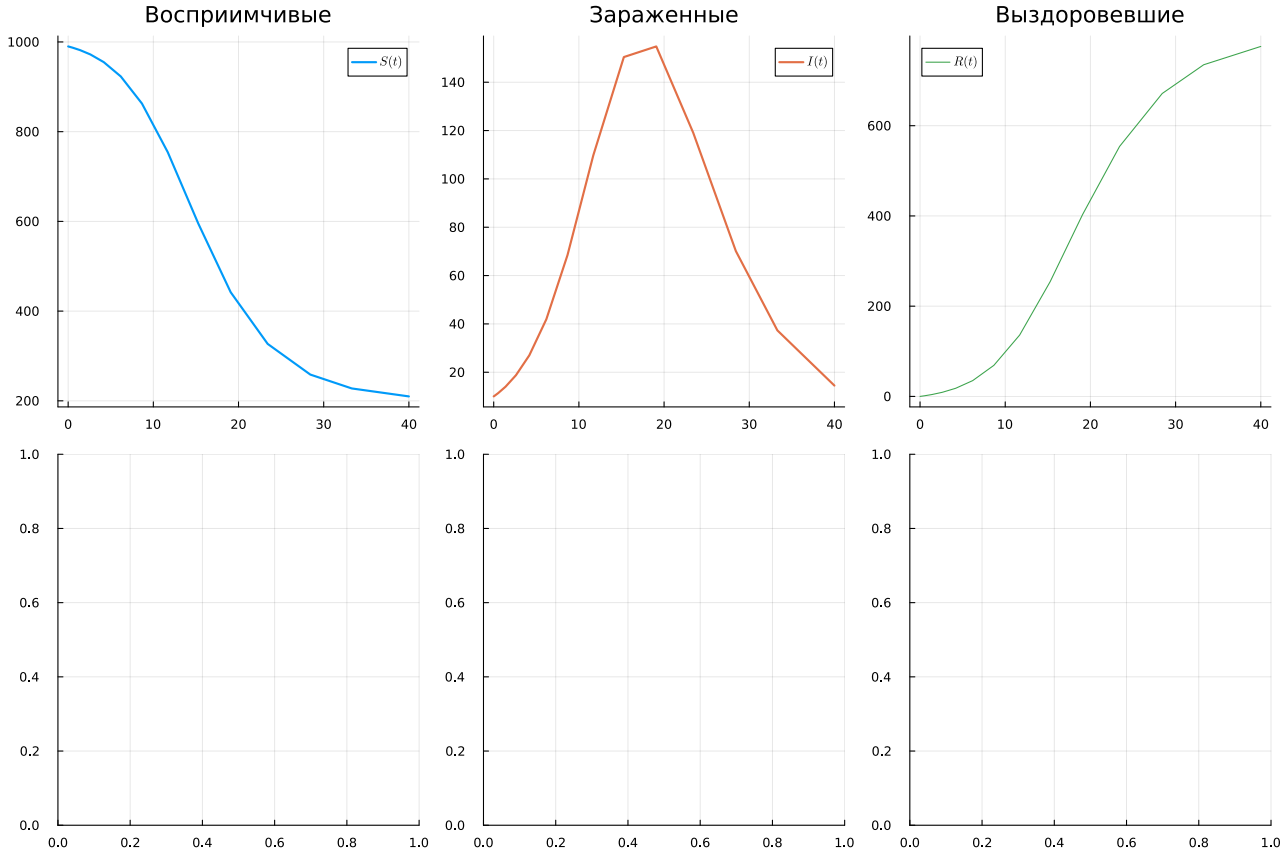
Компактный график всех кривых в одной панели

```
plt7 = plot(layout=(2, 3), size=(1200, 800))
```



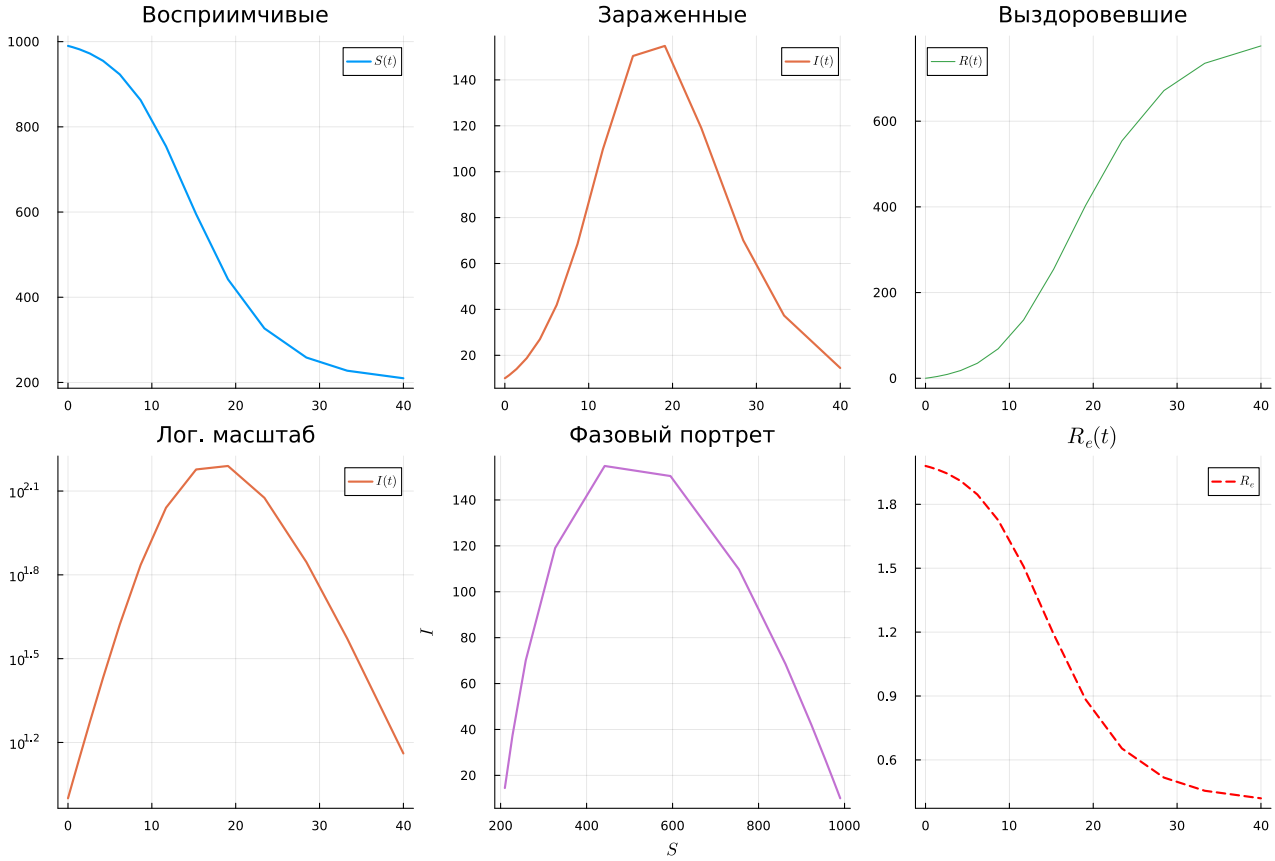
Верхний ряд

```
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1,
linewidth=2, title="Восприимчивые")
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2,
linewidth=2, title="Зараженные")
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3,
inewidth=2, title="Выздоровевшие")
```



Нижний ряд

```
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2,
linewidth=2,
yscale=:log10, title="Лог. масштаб")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4, linewidth=2,
title="Фазовый портрет", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green,
linewidth=2,
title=L"R_e(t)", hline=[1.0], linestyle=:dash, linecolor=:red)
```



Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))
```

```
"/home/vparbatova/work/study/2025-2026/Иммитационное моделирование/immod/la
```

Бенчмарк для оценки производительности

```
println("\nБенчмарк решения:")
@benchmark solve(prob_ode, dt = δt)
```

```
Бенчмарк решения:

BenchmarkTools.Trial: 10000 samples with 4 evaluations per sample.
 Range (min … max):   7.023 μs …  12.752 ms  ┊ GC (min … max): 0.00% … 99.6
 Time  (median):     10.438 μs               ┊ GC (median):    0.00%
 Time  (mean ± σ):   15.166 μs ± 138.495 μs  ┊ GC (mean ± σ):  8.38% ±  1.0
```

```
  7.02 μs        Histogram: log(frequency) by time      56.1 μs <

 Memory estimate: 17.53 KiB, allocs estimate: 334.
```

Дополнительный анализ

```
println("\n=== АНАЛИЗ РЕЗУЛЬТАТОВ ===")
println("Общая численность популяции (контроль): N = ",
round(df_ode.N[1], digits=1))
println("Пиковое число зараженных: I_max = ", round(peak_value,
digits=1))
println("Время достижения пика: t_peak = ", round(peak_time,
digits=1), " дней")
println("Итоговое число переболевших: R(∞) = ", round(df_ode.R[end],
digits=1))
println("Доля переболевших: ", round(df_ode.R[end]/df_ode.N[1]*100,
digits=1), "%")
if R0 > 1
println("\nТеоретический анализ:")
println(" - Порог коллективного иммунитета: ",
round((1-1/R0)*100, digits=1), "%")
println(" - Теоретический пик при S/N = 1/R0 = ", round(1/R0,
```

```
digits=3))
end
```

```
=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
 - Порог коллективного иммунитета: 50.0%
 - Теоретический пик при S/N = 1/R0 = 0.5
```

SIR — это классическая компартментная модель в эпидемиологии, описывающая распространение инфекционного заболевания в популяции. Название происходит от трёх состояний индивидов:

- **S (Susceptible)** — восприимчивые к болезни (здоровые, но могут заразиться)
- **I (Infectious)** — инфицированные (заразны и могут передавать болезнь)
- **R (Recovered)** — выздоровевшие (приобрели иммунитет и больше не участвуют в распространении)

Модель описывается следующей системой ОДУ:

$$\begin{cases} \dfrac{dS}{dt} = -\beta \cdot S \cdot I \\ \dfrac{dI}{dt} = \beta \cdot S \cdot I - \gamma \cdot I \\ \dfrac{dR}{dt} = \gamma \cdot I \end{cases}$$

Параметры модели - β — коэффициент передачи инфекции (скорость заражения) - γ — коэффициент выздоровления (обратная величина средней продолжительности болезни)

Базовое репродуктивное число Важнейший эпидемиологический показатель:

$$R_0 = \frac{\beta \cdot S_0}{\gamma}$$

- $R_0 > 1$ — эпидемия разрастается

- $R_0 < 1$ — эпидемия затухает

Основные свойства - Полное население $N = S + I + R$ остаётся постоянным - Модель не учитывает рождаемость, смертность (кроме выздоровления) и миграцию - Подходит для острых инфекций с пожизненным иммунитетом (корь, грипп и др.)

На прикрепленном скриншоте виден анализ результатов выполнения предложенного кода (рис. 4.3).

```
julia> include("scripts/sir_ode.jl")
Параметры модели SIR:
β (вероятность заражения) = 0.05
c (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R0 = c * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 =
0.0

Бенчмарк решения:

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
 - Порог коллективного иммунитета: 50.0%
 - Теоретический пик при S/N = 1/R0 = 0.5
```

Рисунок 4.3: Анализ результатов

В коде также прописано создание графиков для наглядности результатов. Все они на одном изображении приложенны ниже (рис. 4.4).

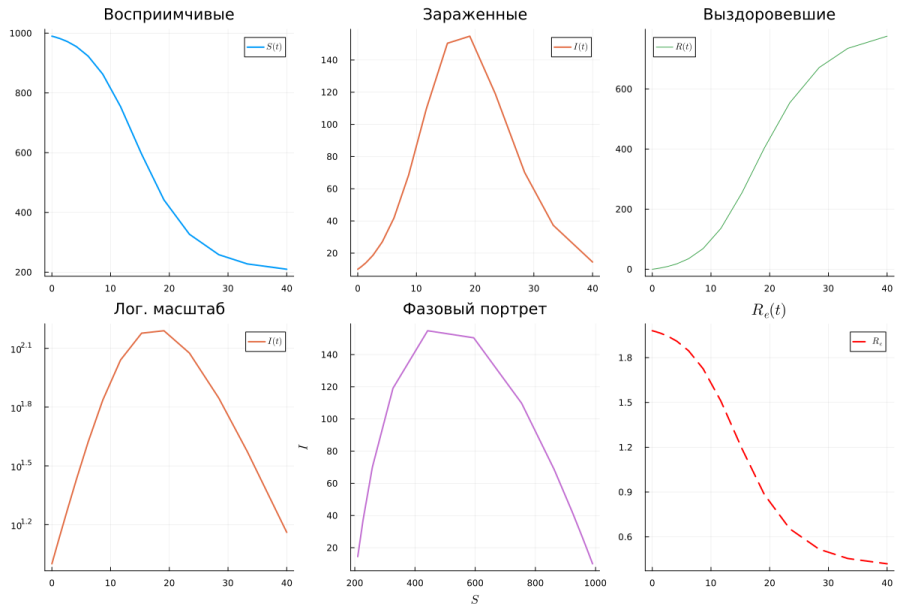


Рисунок 4.4: Графики SIR

С помощью файла, созданного в хоже выполнения лабораторной работы номер 1, создаю читсый код, markdown и jupyter notebook одной командой (рис. 4.5).

```
[vparbatova@vparbatova project]$ julia --project=. scripts/tangle.jl scripts/sir_ode.jl
Генерация из: scripts/sir_ode.jl
[ Info: generating plain script file from `~/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/scripts/sir_ode.jl`
[ Info: writing result to `~/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/scripts/sir_ode/sir_ode.jl`
✓ Чистый скрипт: /home/vparbatova/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/scripts/sir_ode/sir_ode.jl
[ Info: generating markdown page from `~/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/scripts/sir_ode.jl`
[ Info: writing result to `~/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/markdown/sir_ode/sir_ode.qmd`
✓ Quarto: /home/vparbatova/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/markdown/sir_ode/sir_ode.qmd
[ Info: generating notebook from `~/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/scripts/sir_ode.jl`
[ Info: writing result to `~/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/notebooks/sir_ode/sir_ode.ipynb`
✓ Notebook: /home/vparbatova/work/study/2025-2026/Иммитационное моделирование/immod/labs/lab02/project/notebooks/sir_ode/sir_ode.ipynb
```

Рисунок 4.5: Создание файлов

Выполняю jupyter notebook, получаю все те же результаты (рис. 4.6).

```
[25]: println("\n=== АНАЛИЗ РЕЗУЛЬТАТОВ ===")
      println("Общая численность популяции (контроль): N = ",
      round(df_ode.N[1], digits=1))
      println("Пиковое число зараженных: I_max = ", round(peak_value,
      digits=1))
      println("Время достижения пика: t_peak = ", round(peak_time,
      digits=1), " дней")
      println("Итоговое число переболевших: R(∞) = ", round(df_ode.R[end],
      digits=1))
      println("Доля переболевших: ", round(df_ode.R[end]/df_ode.N[1]*100,
      digits=1), "%")
      if R0 > 1
      println("\nТеоретический анализ:")
      println(" - Порог коллективного иммунитета: ",
      round((1-1/R0)*100, digits=1), "%")
      println(" - Теоретический пик при S/N = 1/R0 = ", round(1/R0,
      digits=3))
      end

      === АНАЛИЗ РЕЗУЛЬТАТОВ ===
      Общая численность популяции (контроль): N = 1000.0
      Пиковое число зараженных: I_max = 154.8
      Время достижения пика: t_peak = 19.1 дней
      Итоговое число переболевших: R(∞) = 775.7
      Доля переболевших: 77.6%

      Теоретический анализ:
       - Порог коллективного иммунитета: 50.0%
       - Теоретический пик при S/N = 1/R0 = 0.5

[ ]:
```

Рисунок 4.6

4.3 Модель Лотки-Вольтерры

```
using DrWatson
@quickactivate "project"
using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"/home/vparbatova/work/study/2025-2026/Иммитационное моделирование/immod/la
```

Описание модели Лотки-Вольтерры

```
"""
Модель Лотки-Вольтерры (хищник-жертва)
Система уравнений:
dx/dt = αx - βxy # Изменение популяции жертв
dy/dt = δxy - γy # Изменение популяции хищников
Где:
x - популяция жертв (например, зайцы)
y - популяция хищников (например, лисы)
α - естественный прирост жертв (в отсутствие хищников)
β - коэффициент поедания жертв хищниками
δ - коэффициент прироста хищников за счет поедания жертв
γ - естественная смертность хищников (в отсутствие жертв)
"""
function lotka_volterra!(du, u, p, t)
x, y = u # x - жертвы, y - хищники
α, β, δ, γ = p # параметры модели
@inbounds begin
du[1] = α*x - β*x*y # уравнение для жертв
du[2] = δ*x*y - γ*y # уравнение для хищников
end
nothing
end
```

```
Main.Notebook.lotka_volterra!
```

Параметры модели и начальные условия Классические параметры из литературы

```
p_lv = [0.1, # α: скорость размножения жертв
0.02, # β: скорость поедания жертв хищниками
0.01, # δ: коэффициент конверсии пищи (жертв) в хищников
0.3] # γ: смертность хищников
```

```
4-element Vector{Float64}:
 0.1
 0.02
 0.01
 0.3
```

Начальные условия: [жертвы, хищники]

```
u0_lv = [40.0, 9.0] # начальная популяция
```

```
2-element Vector{Float64}:
 40.0
  9.0
```

Временные параметры

```
tspan_lv = (0.0, 200.0) # длительность симуляции
dt_lv = 0.01 # шаг интегрирования
```

```
0.01
```

Создание и решение задачи

```
prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv,
dt = dt_lv,
Tsit5(), # Метод 5-го порядка
reltol=1e-8, # Относительная точность
abstol=1e-10, # Абсолютная точность
saveat=0.1, # Сохраняем каждые 0.1 единицы времени
dense=true # Включаем плотный вывод для интерполяции
)
```

```
┌ Warning: Verbosity toggle: dense_output_saveat
│  Dense output is incompatible with saveat. Please use the SavingCallback
└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/YaLY7/src/solve

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 2001-element Vector{Float64}:
   0.0
   0.1
   0.2
   0.3
   0.4
   0.5
   0.6
   0.7
   0.8
   0.9
   ⋮
 199.2
 199.3
 199.4
 199.5
 199.6
 199.7
 199.8
 199.9
 200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.67773201890398, 9.088990237207573]
 [39.3511378950319, 9.1758827263069]
 [39.02053587808413, 9.260562174341601]
```

```
 [38.6862482028163, 9.342916311941519]
 [38.348600187590776, 9.422836265416295]
 [38.007919320048416, 9.500216918705295]
 [37.664534369324684, 9.574957250331227]
 [37.318774466549755, 9.646960666854303]
 [36.970968217675335, 9.71613530831863]
 
 [17.824353144929567, 5.50801931850314]
 [17.807442037033947, 5.441315154549741]
 [17.79290768632291, 5.375334300035647]
 [17.780718473177817, 5.310082577781689]
 [17.770843588537023, 5.245565084818681]
 [17.763253013148027, 5.181786245480113]
 [17.757917517460715, 5.118749811715716]
 [17.75480865796634, 5.056458873592463]
 [17.75389874795947, 4.994915937053484]
```

Подготовка данных

```julia
df_lv = DataFrame()
df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u] # жертвы
df_lv[!, :predator] = [u[2] for u in sol_lv.u] # хищники
```

```
2001-element Vector{Float64}:
 9.0
 9.088990237207573
 9.1758827263069
 9.260562174341601
 9.342916311941519
 9.422836265416295
 9.500216918705295
```

```
 9.574957250331227
 9.646960666854303
 9.71613530831863
 ⋮
 5.50801931850314
 5.441315154549741
 5.375334300035647
 5.310082577781689
 5.245565084818681
 5.181786245480113
 5.118749811715716
 5.056458873592463
 4.994915937053484
```

Рассчет производных для анализа

```
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey .* df_
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .-
p_lv[4] .* df_lv.predator
```

```
2001-element Vector{Float64}:
  0.9000000000000004
  0.8796081183812947
  0.8580494468233706
  0.8353523334487725
  0.8115489002366512
  0.7866749261309169
  0.7607697060795098
  0.733875889299783
  0.7060392940838303
  0.6773087043292239
  ⋮
```

```
 -0.6706369809299996
 -0.6634355041661331
 -0.6561720201740996
 -0.6488539394858811
 -0.6414883588875672
 -0.6340820722588977
 -0.6266415740250623
 -0.6191730642026372
 -0.6116824631058786
```

Вывод информации о модели

```
println("="^60)
println("Модель Лотки-Вольтерры (хищник-жертва)")
println("="^60)
println("\nПараметры модели:")
println("α (скорость размножения жертв) = ", p_lv[1])
println("β (скорость поедания жертв) = ", p_lv[2])
println("δ (коэффициент конверсии) = ", p_lv[3])
println("γ (смертность хищников) = ", p_lv[4])
println("\nНачальные условия:")
println("Жертвы (x0) = ", u0_lv[1])
println("Хищники (y0) = ", u0_lv[2])
```

```
============================================================
Модель Лотки-Вольтерры (хищник-жертва)
============================================================


Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
```

```
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0
```

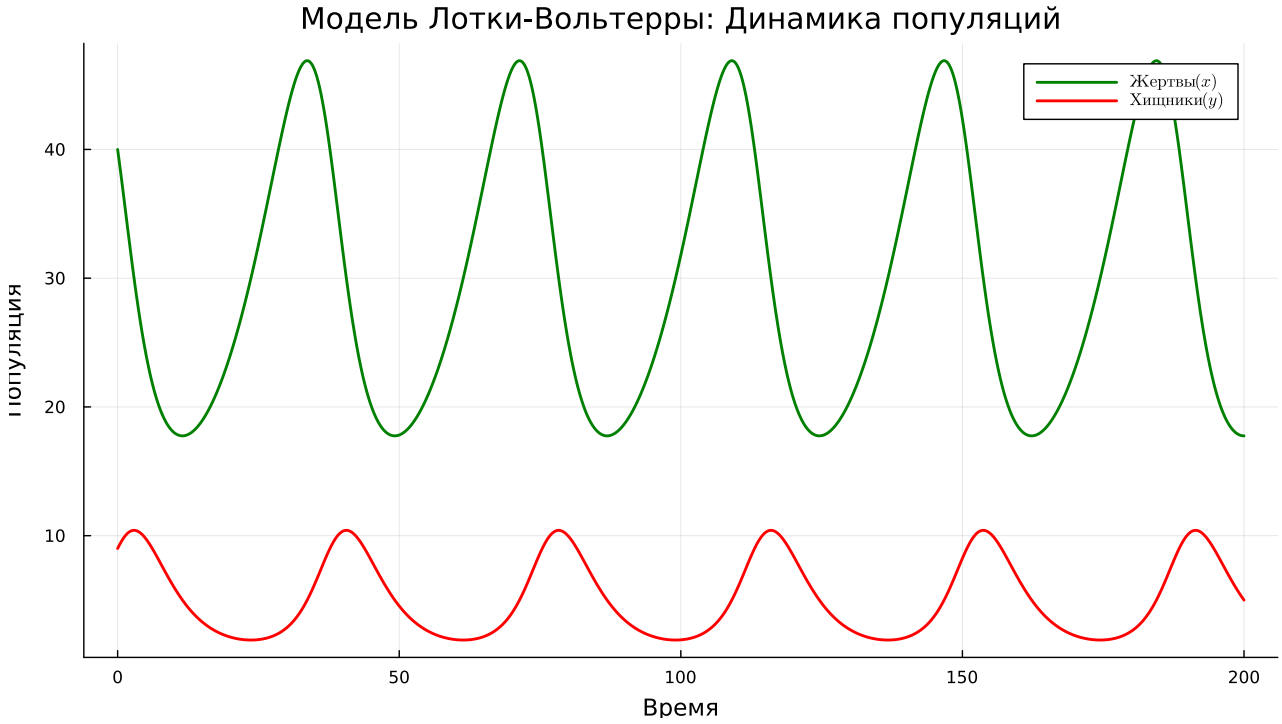
Стационарные точки (нулевые изоклины)

```
x_star = p_lv[4] / p_lv[3] # стационарная точка для жертв
y_star = p_lv[1] / p_lv[2] # стационарная точка для хищников
println("\nСтационарные точки (положения равновесия):")
println("x* = γ/δ = ", round(x_star, digits=3))
println("y* = α/β = ", round(y_star, digits=3))
```

```
Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0
```

Построение графиков График 1: Динамика популяций во времени

```
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
label=[L"Жертвы (x)" L"Хищники (y)"],
xlabel="Время",
ylabel="Популяция",
title="Модель Лотки-Вольтерры: Динамика популяций",
linewidth=2,
legend=:topright,
grid=true,
size=(900, 500),
color=[:green :red])
```



Добавление стационарных уровней

```
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5,
label="x* (равновесие жертв)")
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5,
label="y* (равновесие хищников)")
```

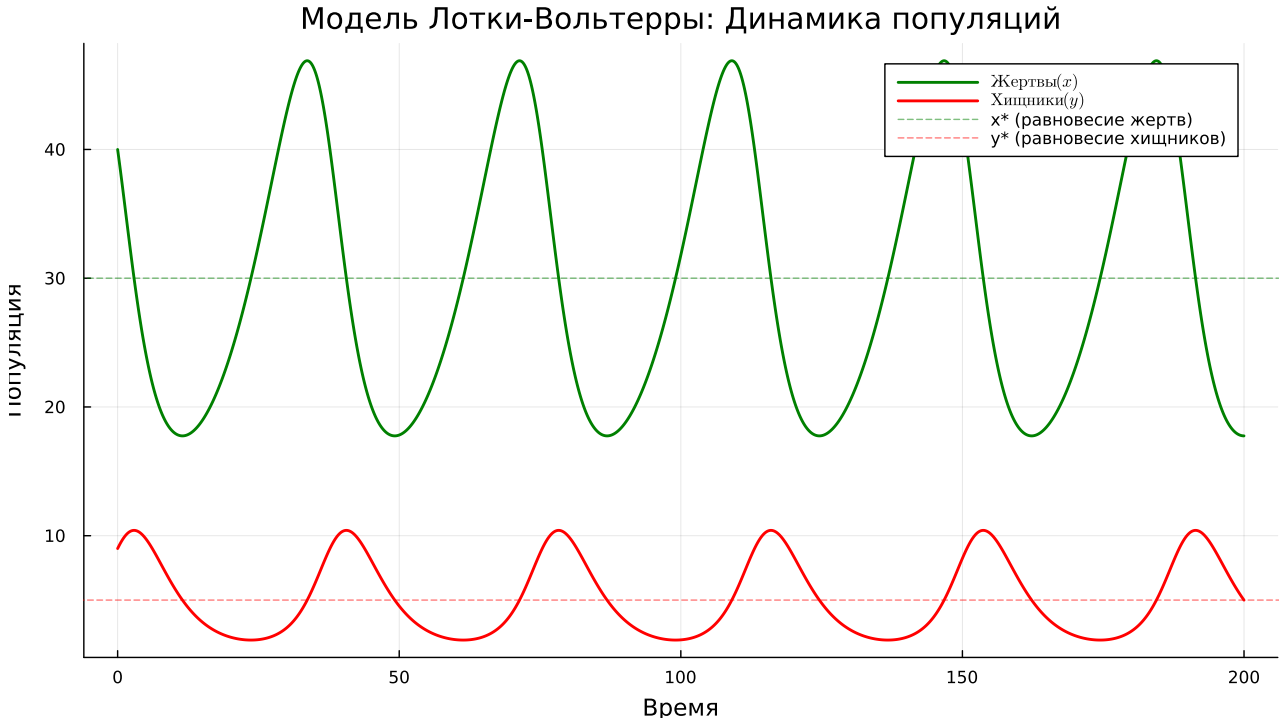
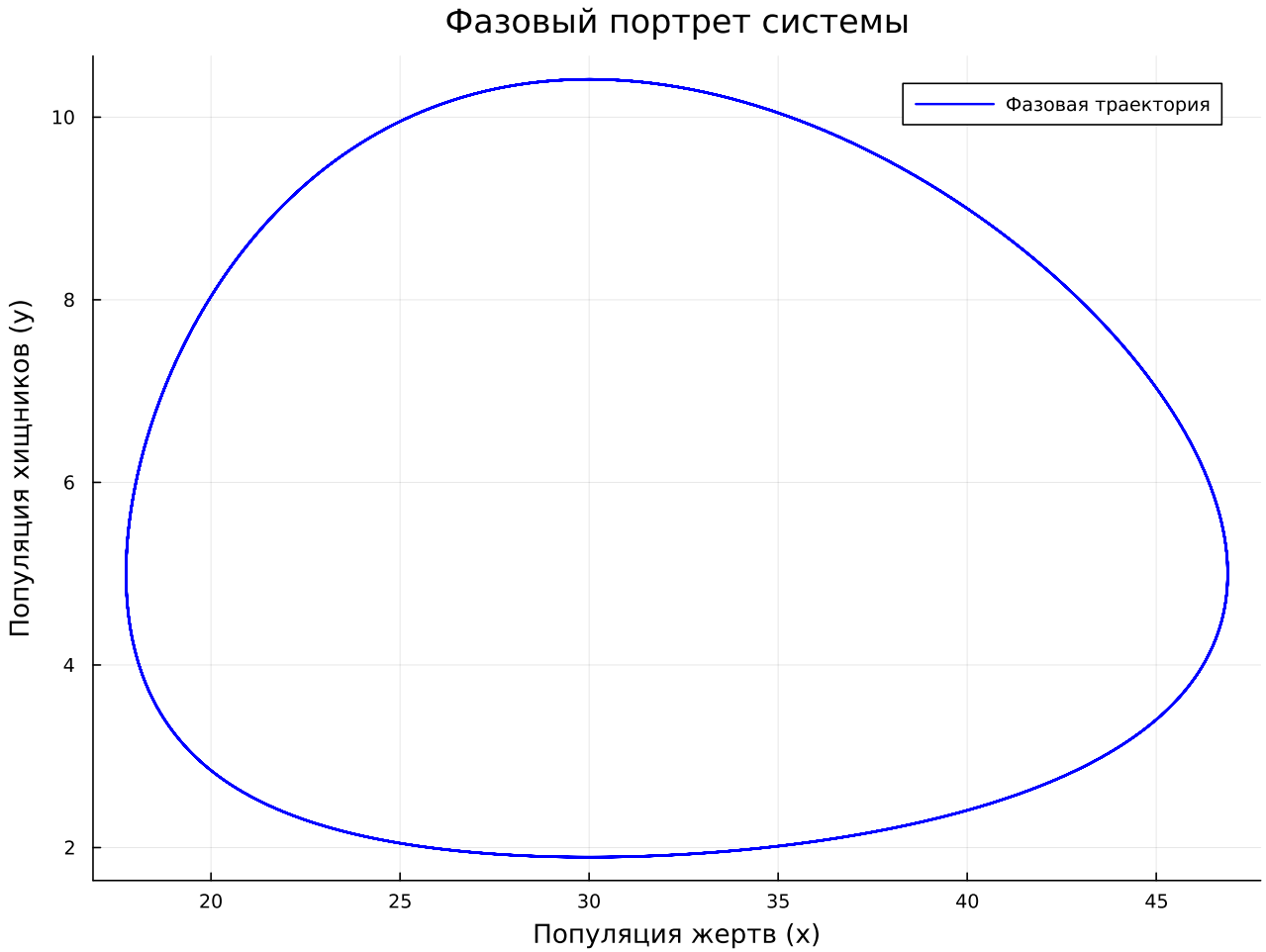


График 2: Фазовый портрет (хищники vs жертвы)

```
plt2 = plot(df_lv.prey, df_lv.predator,
label="Фазовая траектория",
xlabel="Популяция жертв (x)",
ylabel="Популяция хищников (y)",
title="Фазовый портрет системы",
color=:blue,
linewidth=1.5,
grid=true,
size=(800, 600),
legend=:topright)
```

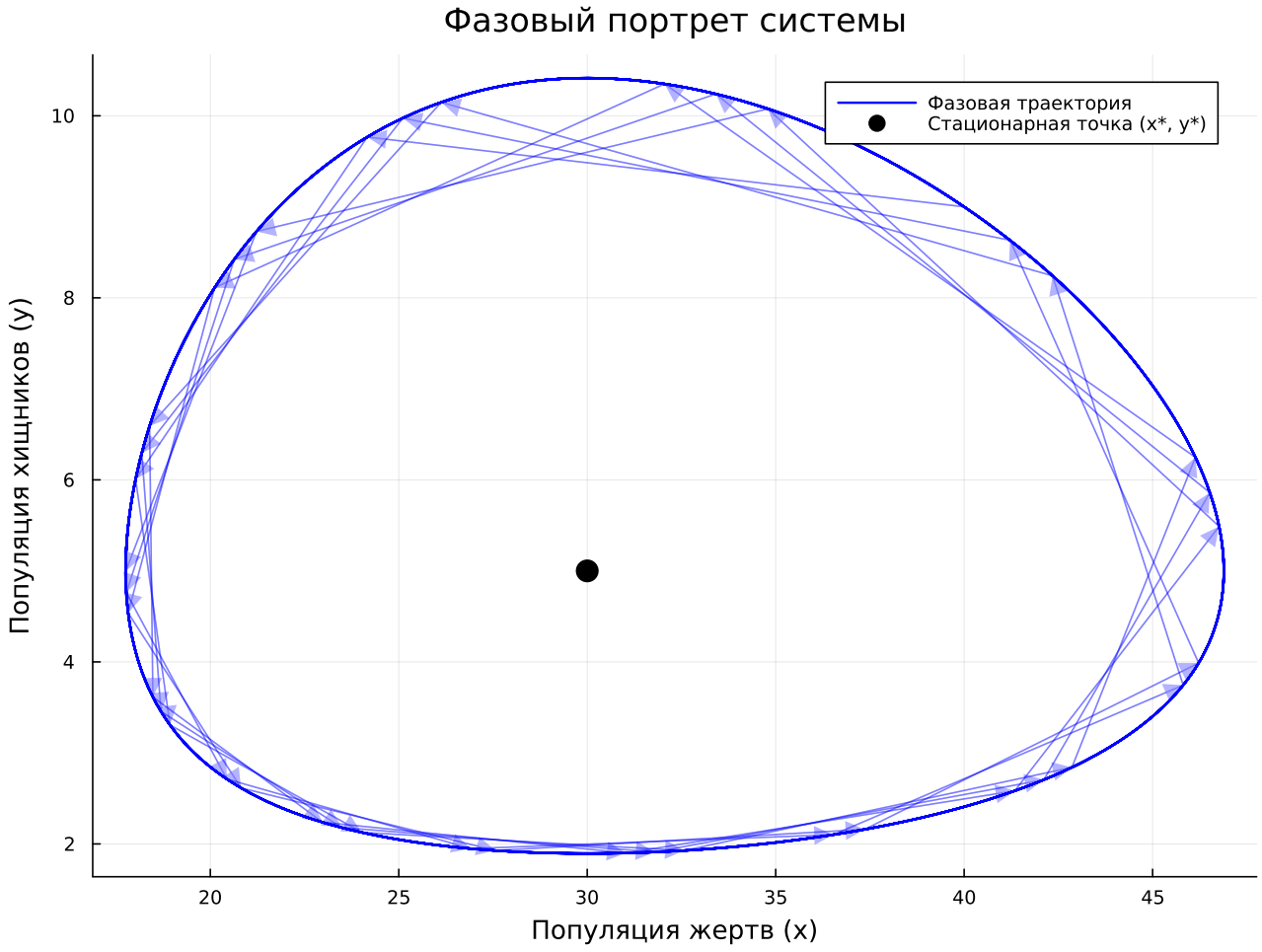


Добавление стрелок направления на фазовом портрете

```
step = 50 # шаг для отображения стрелок
for i in 1:step:length(df_lv.prey)-step
plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
[df_lv.predator[i], df_lv.predator[i+step]],
arrow=:closed, color=:blue, alpha=0.3, label=false)
end
```

Добавление стационарной точки

```
scatter!(plt2, [x_star], [y_star],
color=:black, markersize=8, label="Стационарная точка (x*, y*)")
```



Изоклины (нулевого роста)

```
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range) # y-изоклина (dy/dt = 0)
plot!(plt2, x_range, y_nullcline,
color=:red, linestyle=:dash, linewidth=1.5, label="Изоклина
хищников (dy/dt=0)")
y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range))) # x-изоклина (d
plot!(plt2, x_nullcline, y_range,
color=:green, linestyle=:dash, linewidth=1.5, label="Изоклина
жертв (dx/dt=0)")
```

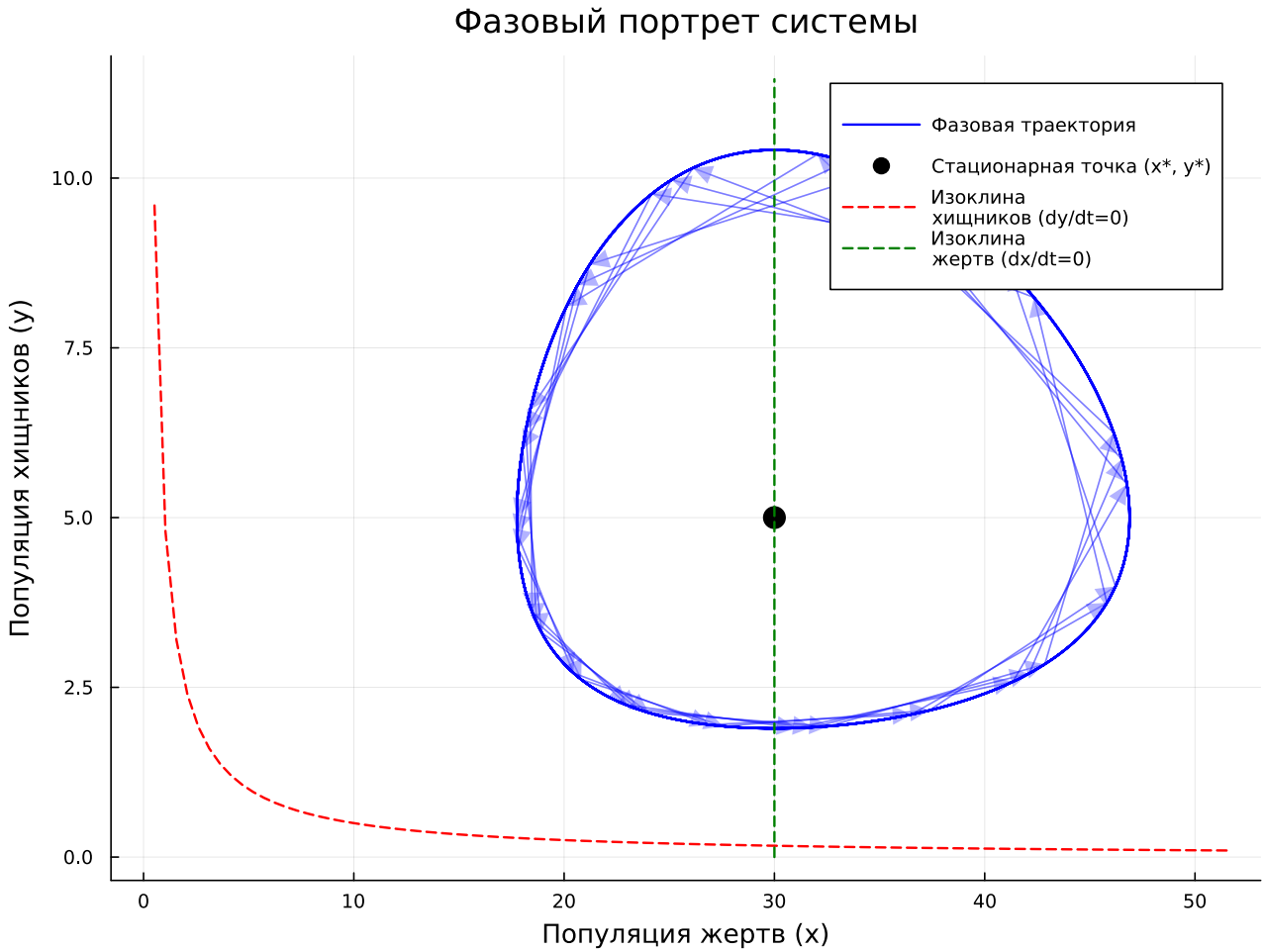


График 3: Производные (скорости изменения)

```
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
label=[L"dx/dt" L"dy/dt"],
xlabel="Время",
ylabel="Скорость изменения",
title="Производные популяций",
linewidth=1.5,
legend=:topright,
grid=true,
size=(900, 400),
color=[:green :red])
hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3,
label=false)
```

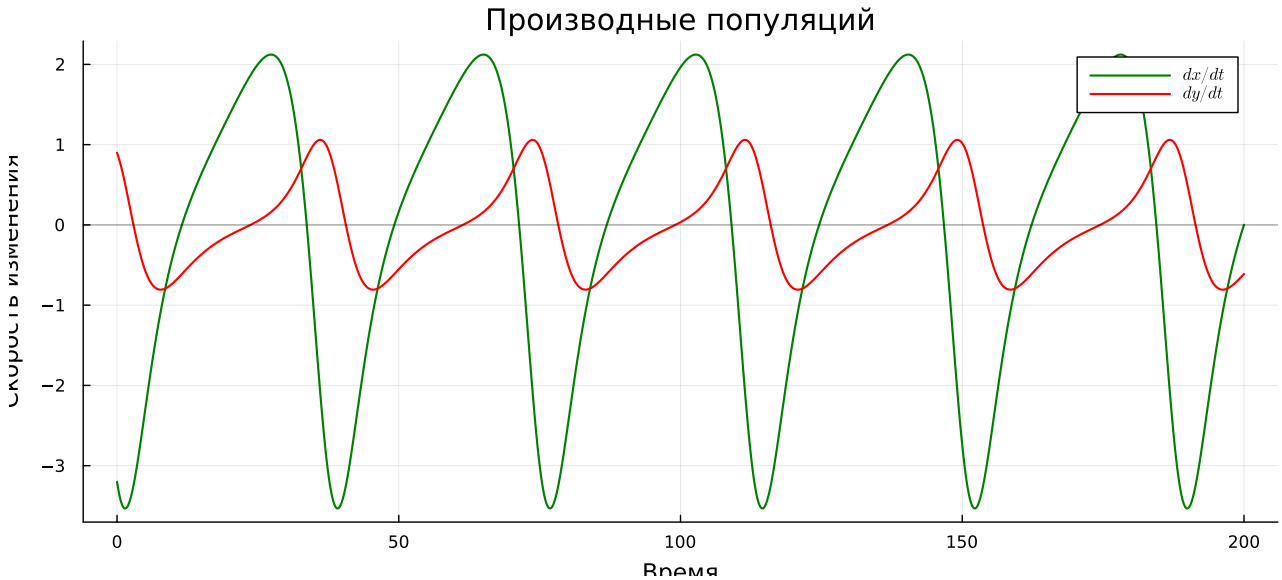


График 4: Относительные изменения (в %)

```
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predator .* 10
plt4 = plot(df_lv.t, [df_lv.prey_pct_change, df_lv.predator_pct_change],
label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
xlabel="Время",
ylabel="Относительное изменение, %",
title="Относительные темпы роста",
```

```
linewidth=1.5,
legend=:topright,
grid=true,
size=(900, 400),
color=[:green :red])
```

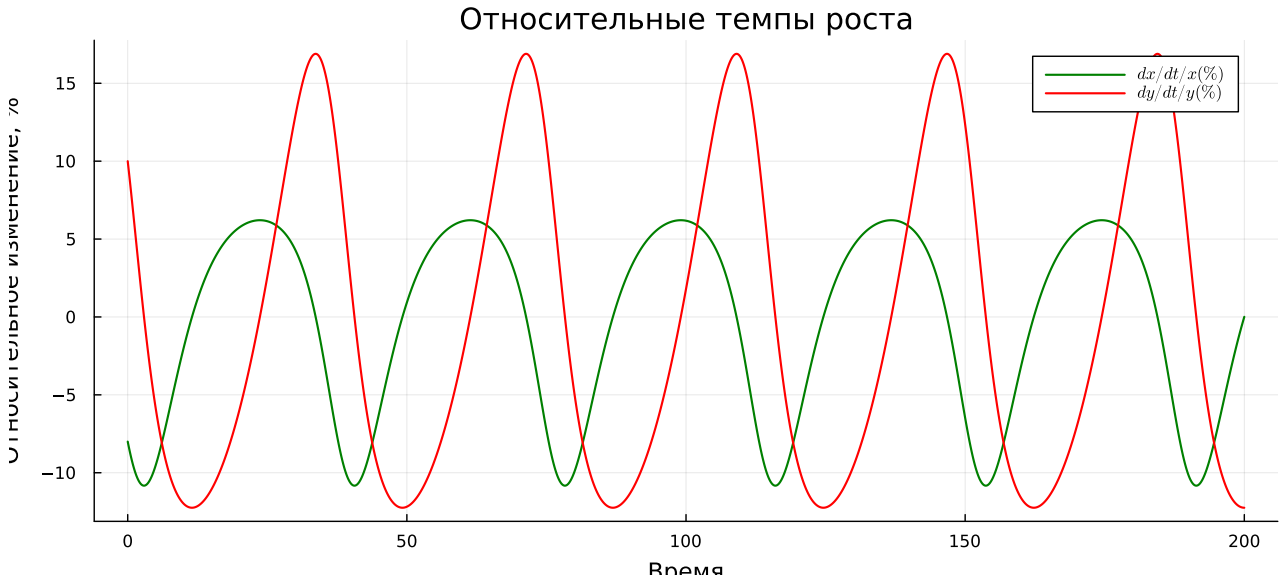


График 5: Спектральный анализ (быстрое преобразование Фурье)

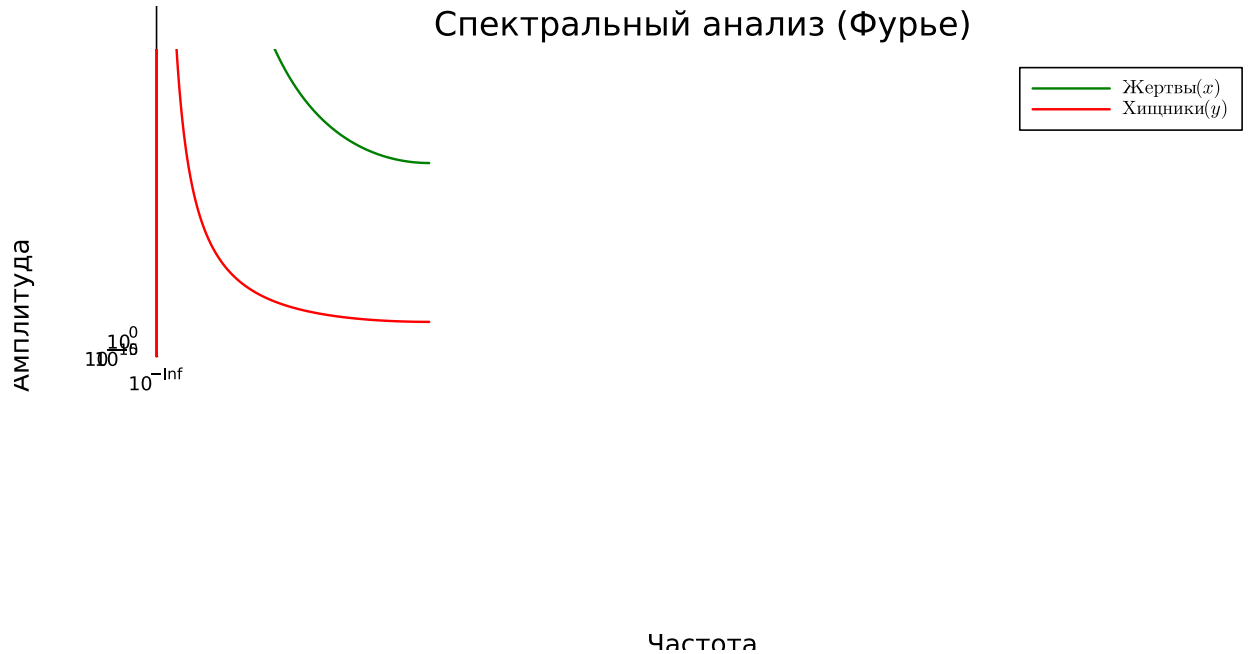
```
function compute_fft(signal, dt)
n = length(signal)
spectrum = abs.(rfft(signal))
freq = rfftfreq(n, 1/dt)
spectrum = max.(spectrum, 1e-16)
return freq, spectrum
end
```

```
compute_fft (generic function with 1 method)
```

Вычисление периодов колебаний

```
freq_prey, spectrum_prey = compute_fft(df_lv.prey .-mean(df_lv.prey), dt_lv
freq_predator, spectrum_predator = compute_fft(df_lv.predator .-mean(df_lv.
plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
```

```
label=[L"Жертвы (x)" L"Хищники (y)"],
xlabel="Частота",
ylabel="Амплитуда",
title="Спектральный анализ (Фурье)",
linewidth=1.5,
xscale=:log10,
yscale=:log10,
legend=:topright,
grid=true,
size=(800, 400),
color=[:green :red])
```



Нахождение доминирующих частот

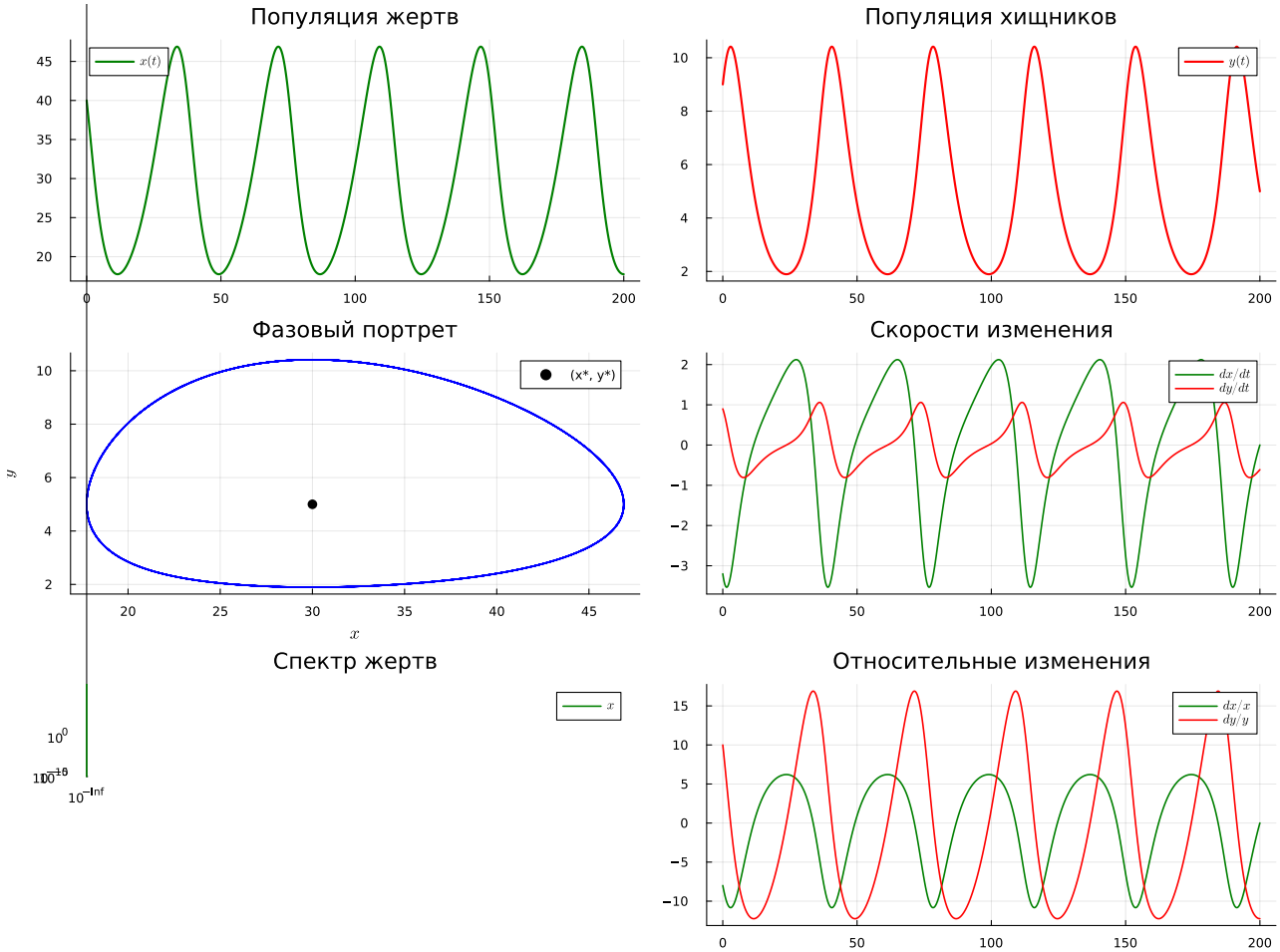
```
if length(spectrum_prey) > 0
idx_prey = argmax(spectrum_prey[2:end]) + 1 # пропускаем нулевую частоту
dominant_freq_prey = freq_prey[idx_prey]
period_prey = 1/dominant_freq_prey
println("\nДоминирующая частота колебаний жертв: ",
round(dominant_freq_prey, digits=4), " Гц")
```

```
println("Период колебаний жертв: ", round(period_prey,
digits=2), " единиц времени")
end
```

```
Доминирующая частота колебаний жертв: 0.2499 Гц
Период колебаний жертв: 4.0 единиц времени
```

График 6: Компактная панель всех графиков

```
plt6 = plot(layout=(3, 2), size=(1200, 900))
plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green,
linewidth=2,
title="Популяция жертв", grid=true)
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red,
linewidth=2,
title="Популяция хищников", grid=true)
plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue,
linewidth=1.5,
title="Фазовый портрет", xlabel=L"x", ylabel=L"y", grid=true)
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5,
label="(x*, y*)")
plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5,
title="Скорости изменения", grid=true, legend=:topright)
plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green,
linewidth=1.5,
title="Спектр жертв", xscale=:log10, yscale=:log10, grid=true)
plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change, df_lv.predator_pct_change],
label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
title="Относительные изменения", grid=true, legend=:topright)
```



Анализ результатов

```
println("\n" * "="^60)
println("Анализ результатов")
println("="^60)
println("\nОсновные статистики:")
println("Жертвы: min = ", round(minimum(df_lv.prey), digits=2),
", max = ", round(maximum(df_lv.prey), digits=2),
", mean = ", round(mean(df_lv.prey), digits=2))
println("Хищники: min = ", round(minimum(df_lv.predator), digits=2),
", max = ", round(maximum(df_lv.predator), digits=2),
", mean = ", round(mean(df_lv.predator), digits=2))
```

```
============================================================
```

```
Анализ результатов
============================================================

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2
```

Упрощенный анализ колебаний без поиска максимумов Вместо сложного анализа сдвига фаз, просто посчитаем основные характеристики Находим время первого пика жертв (простой алгоритм)

```
function find_first_peak(signal, time)
for i in 2:length(signal)-1
if signal[i] > signal[i-1] && signal[i] > signal[i+1]
return time[i], signal[i]
end
end
return NaN, NaN
end
peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey,
df_lv.t)
peak_time_predator, peak_value_predator =
find_first_peak(df_lv.predator, df_lv.t)
if !isnan(peak_time_prey) && !isnan(peak_time_predator)
phase_shift = peak_time_predator - peak_time_prey
println("\nАнализ колебаний:")
println("Первый пик жертв: время = ", round(peak_time_prey,
digits=2),
", значение = ", round(peak_value_prey, digits=2))
println("Первый пик хищников: время = ",
round(peak_time_predator, digits=2),
", значение = ", round(peak_value_predator, digits=2))
```

```julia
println("Сдвиг фаз (хищники отстают): ", round(phase_shift,
digits=2))
end
```

```
Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8
```

Сохранение графиков

```julia
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for l
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for l
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for l
└ @ Plots ~/.julia/packages/Plots/AE0LB/src/utils.jl:105
```

```
"/home/vparbatova/work/study/2025-2026/Иммитационное моделирование/immod/la
```

Дополнительный анализ: чувствительность к параметрам

```
println("\n\n" * "="^60)
println("Анализ чувствительности")
println("="^60)
```

```
============================================================
Анализ чувствительности
============================================================
```

Функция для анализа влияния параметров

```
function analyze_parameter_sensitivity(param_index, values, param_name)
    println("\nАнализ чувствительности к параметру: ", param_name)
    results = []
    for val in values
        p_test = copy(p_lv)
        p_test[param_index] = val
        prob_test = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_test)
        sol_test = solve(prob_test, dt = dt_lv)
        prey_end = sol_test.u[end][1]
        predator_end = sol_test.u[end][2]
        push!(results, (param=val, prey=prey_end, predator=predator_end))
        println(" $(param_name)=$(val): жертвы=$(round(prey_end; digits=2))
    end
    return results
end
```

```
analyze_parameter_sensitivity (generic function with 1 method)
```

Анализ чувствительности к ключевым параметрам

```
if true # Установите в true для выполнения анализа
println("\n1. Влияние скорости размножения жертв (α):")
analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "α")
println("\n2. Влияние смертности хищников (γ):")
analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "γ")
end
println("\nМоделирование завершено успешно!")
```

```
1. Влияние скорости размножения жертв (α):

Анализ чувствительности к параметру: α
 α=0.05: жертвы=16.06, хищники=0.64
 α=0.1: жертвы=17.88, хищники=4.93
 α=0.2: жертвы=26.56, хищники=6.92
 α=0.3: жертвы=33.89, хищники=24.49

2. Влияние смертности хищников (γ):

Анализ чувствительности к параметру: γ
 γ=0.1: жертвы=0.87, хищники=2.07
 γ=0.3: жертвы=17.88, хищники=4.93
 γ=0.5: жертвы=53.5, хищники=9.97
 γ=0.7: жертвы=101.25, хищники=1.4

Моделирование завершено успешно!
```

Модель Лотки-Вольтерры — это классическая математическая модель, описывающая динамику взаимодействия двух биологических популяций: хищников и жертв. Модель была независимо предложена Альфредом Лоткой (1925) и Вито Вольтеррой (1926).

Модель описывается следующей системой ОДУ:

$$\begin{cases} \dfrac{dx}{dt} = \alpha x - \beta xy \\ \\ \dfrac{dy}{dt} = \delta xy - \gamma y \end{cases}$$

где: - x — популяция жертв (например, зайцы) - y — популяция хищников (например, лисы)

Параметры модели - α — коэффициент естественного прироста жертв (в отсутствие хищников) - β — коэффициент смертности жертв от встреч с хищниками (интенсивность хищничества) - δ — коэффициент прироста хищников за счет потребления жертв (эффективность конверсии биомассы) - γ — коэффициент естественной смертности хищников (в отсутствие жертв)

Система имеет две стационарные точки:

1. **Тривиальное равновесие** (вымирание обоих видов):

$$(x^*, y^*) = (0, 0)$$

2. **Нетривиальное равновесие** (сосуществование видов):

$$x^* = \frac{\gamma}{\delta}, \quad y^* = \frac{\alpha}{\beta}$$

Фазовый портрет и изоклины

Изоклины (линии нулевого роста):

- Для жертв ($dx/dt = 0$): $y = \alpha/\beta$ (вертикальная линия на фазовой плоскости)
- Для хищников ($dy/dt = 0$): $x = \gamma/\delta$ (горизонтальная линия)

Характер динамики - В окрестности нетривиальной стационарной точки система демонстрирует **циклические колебания** - На фазовой плоскости траектории представляют собой **замкнутые кривые** (циклы) - Амплитуда

и период колебаний зависят от начальных условий и параметров - Хищники следуют за изменениями популяции жертв с **фазовым сдвигом**

Основные свойства - Полная энергия системы (в консервативном приближении) сохраняется - Система является **консервативной** (имеет первый интеграл):

$$V = \delta x + \beta y - \gamma \ln x - \alpha \ln y = \text{const}$$

- Модель не учитывает внутривидовую конкуренцию, ограниченность ресурсов и другие факторы

Провожу все те же манипуляции для второй модели и выполняю созданный jupyter notebook (рис. 4.7).

```
============================================================
Анализ результатов
============================================================

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2
```

Упрощенный анализ колебаний без поиска максимумов Вместо сложного анализа сдвига фаз, просто посчитаем основные характеристики Находим время первого пика жертв (простой алгоритм)

```
[29]: function find_first_peak(signal, time)
for i in 2:length(signal)-1
if signal[i] > signal[i-1] && signal[i] > signal[i+1]
return time[i], signal[i]
end
end
return NaN, NaN
end
peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey,
df_lv.t)
peak_time_predator, peak_value_predator =
find_first_peak(df_lv.predator, df_lv.t)
if !isnan(peak_time_prey) && !isnan(peak_time_predator)
phase_shift = peak_time_predator - peak_time_prey
println("\nАнализ колебаний:")
println("Первый пик жертв: время = ", round(peak_time_prey,
digits=2),
", значение = ", round(peak_value_prey, digits=2))
println("Первый пик хищников: время = ",
round(peak_time_predator, digits=2),
", значение = ", round(peak_value_predator, digits=2))
println("Сдвиг фаз (хищники отстают): ", round(phase_shift,
digits=2))
end

Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8
```

Рисунок 4.7: Анализ результатов

Анализ чувствительности к параметрам (рис. 4.8).

Анализ чувствительности к ключевым параметрам

```
[33]: if true # Установите в true для выполнения анализа
      println("\n1. Влияние скорости размножения жертв (α):")
      analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "α")
      println("\n2. Влияние смертности хищников (γ):")
      analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "γ")
      end
      println("\nМоделирование завершено успешно!")
```

```
1. Влияние скорости размножения жертв (α):

Анализ чувствительности к параметру: α
 α=0.05: жертвы=16.32, хищники=0.62
 α=0.1: жертвы=17.86, хищники=4.96
 α=0.2: жертвы=26.73, хищники=6.87
 α=0.3: жертвы=33.76, хищники=24.48

2. Влияние смертности хищников (γ):

Анализ чувствительности к параметру: γ
 γ=0.1: жертвы=0.87, хищники=2.07
 γ=0.3: жертвы=17.86, хищники=4.96
 γ=0.5: жертвы=54.16, хищники=10.0
 γ=0.7: жертвы=100.93, хищники=1.38

Моделирование завершено успешно!
```

Рисунок 4.8: Анализ чувствительности к параметрам

Графики для наглядности результатов (рис. 4.9).

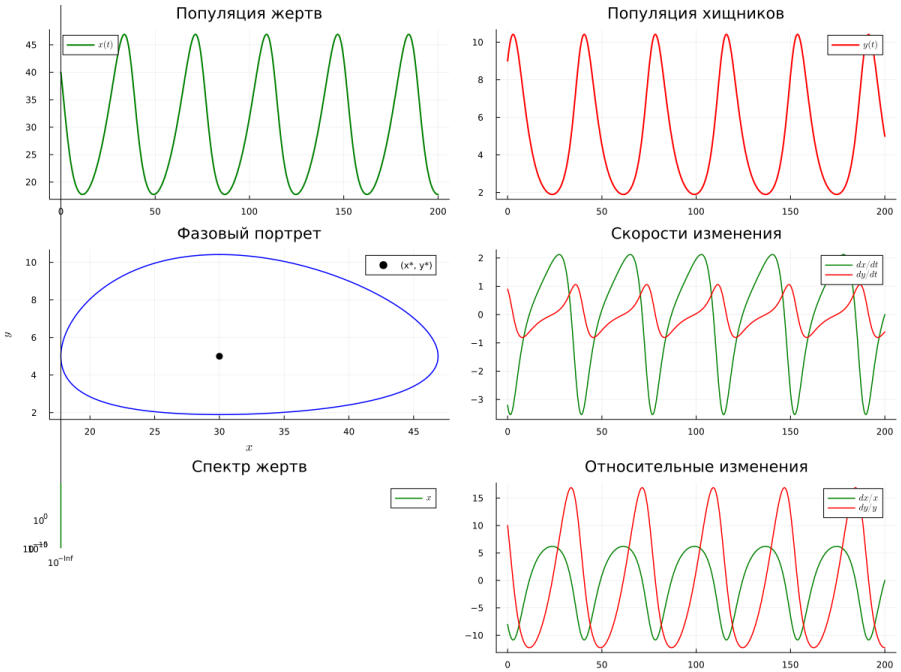


Рисунок 4.9: Графики

5 Выводы

Я ознакомилась с двумя моделями и выполнила предложенный код

Список литературы